

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики
Дисциплина: Компьютерные системы и сети (КСиС)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему
**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ УПРАВЛЕНИЯ БАНКОВСКИМИ
СЧЕТАМИ И КАРТАМИ**
БГУИР КП 6-05 06 12 01 012 ПЗ

Студент

Б. А. Дубинин

Руководитель

С. В. Болтак

СОДЕРЖАНИЕ

Введение	5
1 Постановка задачи	6
1.1 Описание предметной области	6
1.2 Сравнительный анализ существующих решений	6
1.3 Информационная база задачи	8
1.4 Функциональное назначение	9
2 Проектирование задачи	10
2.1 Алгоритм решения задачи	10
2.2 Логическое моделирование	11
2.3 Выбор и обоснование инструментов разработки	11
3 Программная реализация	13
3.1 Физическая структура	13
3.2 Описание разработанных модулей	13
4 Тестирование	18
5 Применение программы	22
5.1 Руководство пользователя	22
Заключение	29
Приложение А(обязательное) Листинг кода с комментариями	30
Приложение Б(обязательное) Схема алгоритма работы системы	57

ВВЕДЕНИЕ

С развитием информационных технологий автоматизация управления финансовыми операциями становится неотъемлемой частью современного банковского бизнеса. В сфере банковского обслуживания актуальным остаётся вопрос создания удобных инструментов для администрирования счетов, проведения переводов и управления платёжными картами. Использование таких решений позволяет существенно повысить прозрачность операций, ускорить обработку запросов и снизить вероятность ошибок, связанных с ручным вводом данных.

Функциональные возможности таких программ постоянно расширяются и улучшаются благодаря конкуренции между их разработчиками и высоким темпам развития и внедрения информационных технологий. Несмотря на то, что программы разных изготовителей базируются на разных технологических решениях, большинство их придерживается общепринятых стандартов и рекомендаций в области обработки и отображения данных.

Однако многие готовые решения административных панелей являются либо избыточно сложными, либо жёстко привязанными к конкретным банковским платформам и не допускают гибкой настройки под нужды конкретного сервиса.

Разработка веб-приложения для администрирования банковских операций ставит своей целью создание простого и удобного инструмента, обеспечивающего централизованное управление счетами, переводами и картами через единый интерфейс. Приложение должно быть интуитивно понятным для операторов и включать функционал для быстрого выполнения задач: создания и просмотра счетов, проведения денежных переводов, формирования выписок по счёту, управления избранными переводами, а также выпуска, блокировки и удаления платёжных карт.

Тема разработки и сама разработка веб-приложений является весьма интересной. Поэтому целью данного курсового проекта является разработка программного средства для управления банковскими счетами и картами.

Внедрение такого решения обеспечит сотрудников инструментом для быстрого и эффективного выполнения административных задач, снизит нагрузку на персонал и повысит общий уровень контроля над проводимыми операциями.

В пояснительной записке рассматриваются задачи, связанные с проектированием и реализацией приложения, включая анализ предметной области, выбор инструментов разработки, проектирование пользовательского интерфейса, а также описание реализации и тестирования программы.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Описание предметной области

Банковская деятельность включает широкий спектр финансовых операций: открытие и ведение счетов, проведение денежных переводов, выпуск и обслуживание платёжных карт, формирование выписок и отчётности. Для выполнения этих задач банки используют специализированные программные средства – административные панели, через которые операторы управляют операциями клиентов.

Оператор – сотрудник банка, работающий с административным интерфейсом системы. В его обязанности входит создание и просмотр банковских счетов, проведение переводов между счетами, управление платёжными картами клиентов, а также формирование выписок по операциям. Оператор не является конечным клиентом банка: он взаимодействует с системой от имени организации, контролируя корректность и полноту проводимых операций.

Банковский счёт – основная сущность системы. Он характеризуется уникальным номером, балансом, валютой и статусом. К одному счёту может быть привязано несколько платёжных карт. Перевод – операция перемещения денежных средств между двумя счетами; фиксируется сумма, дата, счёт-источник и счёт-получатель. Избранный перевод – сохранённый шаблон перевода, который позволяет оператору быстро повторить часто выполняемую операцию без повторного ввода реквизитов.

Платёжная карта привязана к конкретному счёту и может находиться в одном из состояний: активна или заблокирована. Оператор управляет жизненным циклом карты: выпускает, блокирует и удаляет её при необходимости.

Все перечисленные операции выполняются через единый веб-интерфейс, что обеспечивает централизованный контроль над данными и снижает вероятность ошибок при ручной обработке запросов.

1.2 Сравнительный анализ существующих решений

На рынке существует ряд программных решений для управления банковскими операциями. Ниже рассмотрены три наиболее распространённых.

1 Temenos Transact – ведущее цифровое решение для основного банковского обслуживания, используемое более чем 1000 банками в более чем 150 странах. Система способна обрабатывать огромные объёмы операций, поддерживает микросервисную архитектуру и может разворачиваться как локально, так и через облачные сервисы, например, Microsoft Azure. Платформа охватывает розничный, корпоративный банкинг, управление

активами и платежи. Решение интегрируется со сторонними системами (например, через HAProxy для балансировки нагрузки) и аналитическими платформами вроде BankBI для автоматической отчетности. Система обладает широчайшим функциональным охватом, однако её внедрение требует значительных финансовых вложений и длительной настройки под конкретный банк. Для небольших проектов и учебных задач система избыточна по сложности и стоимости. Интерфейс программы представлен на рисунке 1.1.

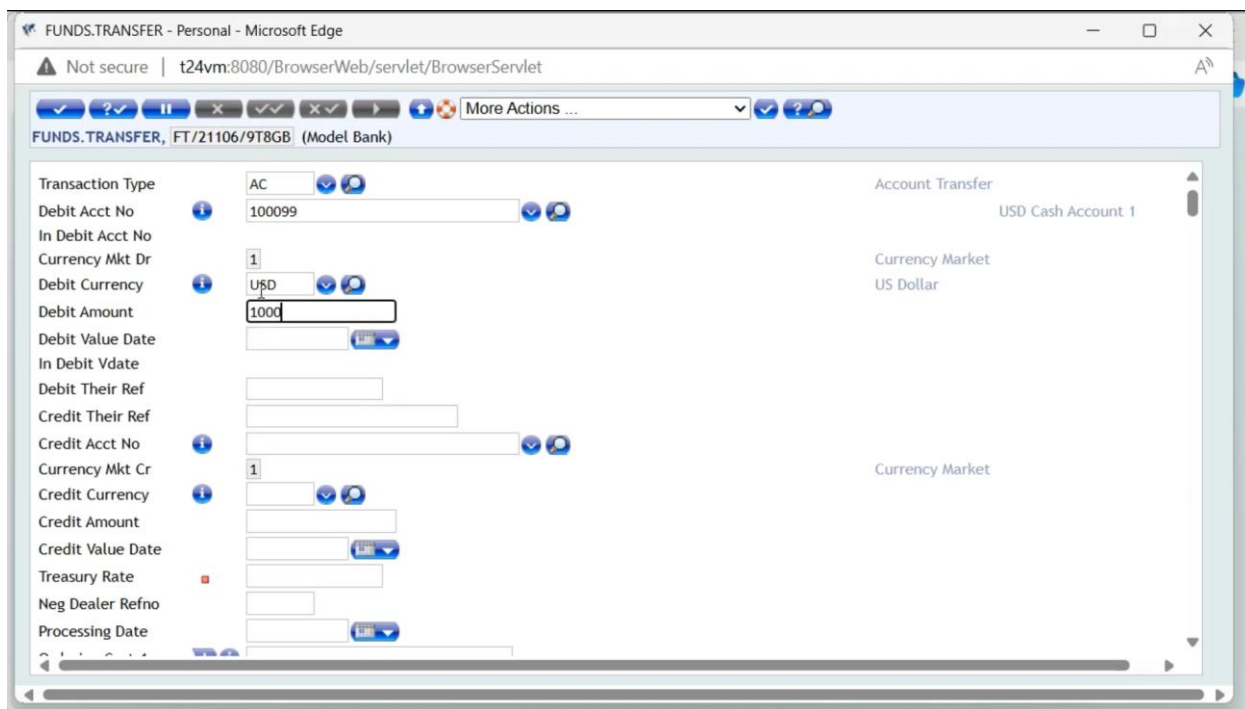


Рисунок 1.1 – Программа Temenos Transact

2 Oracle FLEXCUBE – программная платформа, позволяющая банкам запускать и управлять широким спектром финансовых продуктов и услуг, поддерживающая цифровую трансформацию и обеспечивающая операционную эффективность в крупном масштабе. Несмотря на высокую надёжность, интерфейс системы устарел и требует значительного улучшения с точки зрения современной визуальной эстетики. Кроме того, лицензирование Oracle FLEXCUBE предполагает высокую стоимость, что делает его нецелесообразным для небольших сервисов. Интерфейс программы представлен на рисунке 1.2.

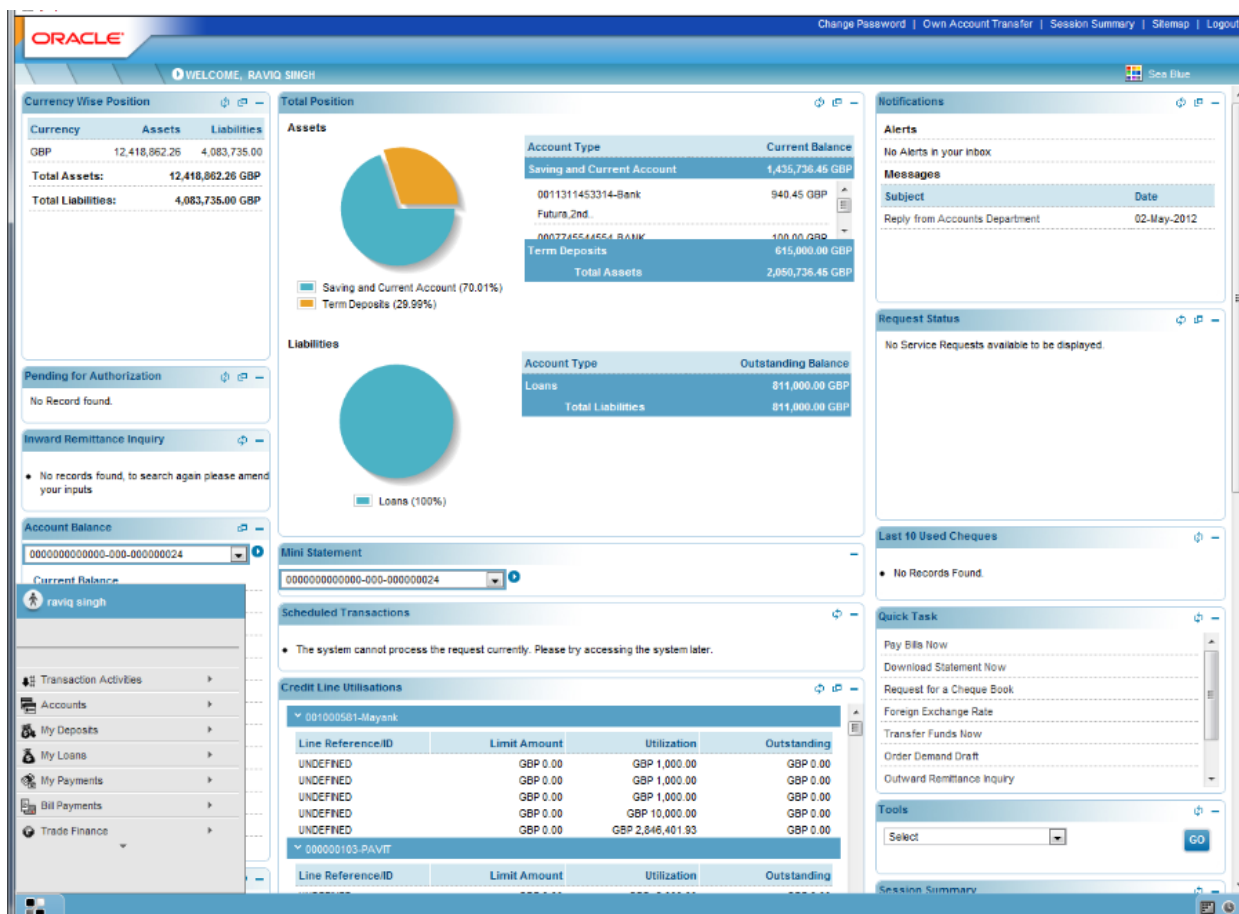


Рисунок 1.2 – Программа Oracle FLEXCUBE

3 Finastra Fusion – решение на основе открытой программной архитектуры и облачной экосистемы, ориентированное на предоставление финансовых технологий нового поколения. Платформа построена по модульному принципу, однако жёстко привязана к собственной экосистеме Finastra и не допускает гибкой интеграции без использования её инфраструктуры.

Сравнение показывает, что все три решения ориентированы на крупные банковские организации и несут высокую технологическую и финансовую нагрузку. Ни одно из них не подходит для создания лёгкого, автономного инструмента с простым веб-интерфейсом, который обеспечивал бы базовые операции: управление счетами, картами, переводами и выписками. Разрабатываемое приложение призвано закрыть именно эту нишу.

1.3 Информационная база задачи

Приложение будет хранить и обрабатывать данные о счетах, картах, переводах и избранных переводах. Все данные хранятся в реляционной базе данных PostgreSQL и связаны между собой через внешние ключи.

Основные категории данных:

1 Счета. Содержат уникальный идентификатор, номер счёта, баланс, валюту и статус (активен / закрыт). Счёт является центральной сущностью – к нему привязаны карты и переводы.

2 Карты. Хранят идентификатор карты, имя владельца, номер карты, идентификатор привязанного счёта, статус (активна / заблокирована) и срок действия. Одному счёту может соответствовать несколько карт.

3 Переводы. Содержат идентификатор перевода, номер счёта-источника, номер счёта-получателя, сумму, валюту и дату проведения операции. Каждый перевод связан ровно с двумя счетами.

4 Избранные переводы. Хранят идентификатор, номер счёта-отправителя, номер счёта-получателя и сумму. Используются для быстрого повторного проведения часто выполняемых переводов.

Связи между сущностями:

- один счёт может иметь несколько карт;
- один счёт может участвовать в нескольких переводах как источник или получатель;
- один избранный перевод ссылается на один счёт-получатель и на один счёт-отправитель.

Такая структура обеспечивает целостность данных и позволяет эффективно формировать выписки по счёту, фильтруя переводы по идентификатору счёта и диапазону дат.

1.4 Функциональное назначение

Система предоставляет оператору следующие возможности:

- создание банковского счёта с указанием валюты и начального баланса;
- просмотр списка всех счетов и информации по каждому из них;
- формирование выписки по счёту за выбранный период;
- выпуск новой платёжной карты с привязкой к счёту и удаление её;
- блокировка и разблокировка карты;
- проведение денежного перевода между двумя счетами;
- просмотр истории переводов по счёту;
- сохранение реквизитов перевода как избранного шаблона;
- просмотр и удаление сохранённых шаблонов;
- быстрое проведение перевода на основе избранного шаблона.

Основная цель приложения – предоставить оператору единый, простой и надёжный инструмент для выполнения повседневных банковских задач без необходимости использования громоздких корпоративных систем.

2 ПРОЕКТИРОВАНИЕ ЗАДАЧИ

2.1 Алгоритм решения задачи

Система построена по клиент-серверной архитектуре. Фронтенд представляет собой набор статических HTML-страниц, которые взаимодействуют с двумя независимыми бэкенд-сервисами через REST API. Оператор работает через браузер: все действия инициируют HTTP-запросы к соответствующему сервису, который обрабатывает их и возвращает ответ в формате JSON. Фронтенд отображает полученные данные без перезагрузки страницы.

Бэкенд разделён на два сервиса по предметному признаку. Сервис аккаунтов и переводов отвечает за управление банковскими счетами, проведение денежных переводов между ними, а также за хранение избранных переводов. Сервис карт отвечает исключительно за управление платёжными картами. Когда для выполнения операции требуются данные из другого сервиса, сервисы обращаются друг к другу синхронно по HTTP.

Алгоритм работы оператора со счетами начинается с открытия соответствующей страницы, на которой фронтенд запрашивает список всех счетов у сервиса аккаунтов и переводов. Для создания нового счёта оператор заполняет форму и отправляет данные; сервис валидирует их и сохраняет в базе данных. Для просмотра выписки по счёту оператор выбирает нужный счёт и задаёт период – сервис возвращает отфильтрованный список переводов, связанных с этим счётом.

Алгоритм проведения перевода состоит из следующих шагов: оператор указывает счёт-источник, счёт-получатель и сумму. Сервис проверяет существование обоих счетов и достаточность баланса на счёте-источнике. При успешной проверке сумма списывается с одного счёта и зачисляется на другой, операция фиксируется в базе данных. При работе с избранными переводами оператор может сохранить реквизиты получателя как шаблон и в дальнейшем использовать его для быстрого заполнения формы перевода.

Алгоритм управления картами начинается с выбора счёта, к которому привязаны карты. Фронтенд запрашивает у сервиса карт список карт по идентификатору счёта. При этом сервис карт обращается к сервису аккаунтов и переводов, чтобы убедиться в существовании указанного счёта. Оператор может выпустить новую карту, заблокировать активную, разблокировать заблокированную или удалить карту. Каждое из этих действий представляет собой отдельный HTTP-запрос к сервису карт, который обновляет состояние карты в своей базе данных.

2.2 Логическое моделирование

Каждый из двух сервисов работает со своей независимой базой данных PostgreSQL. Такое разделение соответствует принципу слабой связанности: каждый сервис полностью владеет своими данными и не обращается напрямую к базе другого сервиса.

База данных сервиса аккаунтов и переводов содержит три таблицы.

Таблица счетов хранит следующие поля: идентификатор счёта, номер счёта, баланс, валюта и статус счёта (активен или закрыт).

Таблица переводов содержит: идентификатор перевода, идентификатор счёта-источника, идентификатор счёта-получателя, сумму, валюту и дату проведения операции. Оба поля счетов являются внешними ключами, ссылающимися на таблицу счетов.

Таблица избранных переводов хранит: идентификатор избранного перевода, идентификатор счёта-отправителя и идентификатор счёта-получателя.

Связи внутри этой базы данных: один счёт может быть источником или получателем во многих переводах; один счёт может быть получателем во многих избранных переводах.

База данных сервиса карт содержит одну таблицу.

Таблица карт хранит: идентификатор карты, номер карты, имя владельца, идентификатор привязанного счёта (хранится как обычное поле, без внешнего ключа – счёт находится в другой базе данных), статус карты (активна или заблокирована) и срок действия.

Связь между базами данных логическая: идентификатор счёта в таблице карт соответствует идентификатору счёта в базе сервиса аккаунтов. Целостность этой связи обеспечивается на уровне бизнес-логики – сервис карт проверяет существование счёта через HTTP-запрос к сервису аккаунтов перед выпуском карты.

2.3 Выбор и обоснование инструментов разработки

Spring Boot и Spring Framework. Для сервиса аккаунтов и переводов выбран Spring Boot – фреймворк, который автоматически настраивает компоненты приложения, управляет зависимостями и предоставляет встроенный веб-сервер. Это позволяет сосредоточиться на логике приложения, а не на инфраструктурных настройках. Для сервиса карт выбран чистый Spring Framework без Spring Boot: такой подход требует ручной настройки всех компонентов, что даёт полный контроль над конфигурацией и позволяет глубже изучить механизмы работы фреймворка. По сравнению с альтернативами, такими как Jakarta EE или Micronaut, Spring обладает

наибольшей распространённостью, обширной документацией и широким сообществом разработчиков.

PostgreSQL. В качестве системы управления базами данных выбрана PostgreSQL – надёжная реляционная СУБД с открытым исходным кодом. Она поддерживает все необходимые типы данных, транзакционную модель и внешние ключи. В отличие от MySQL, PostgreSQL строже соответствует стандарту SQL и лучше обрабатывает параллельные запросы. Встроенная база H2 не подходит, поскольку не обеспечивает постоянного хранения данных в производственной среде.

Spring Data JPA и JDBC Template. В сервисе аккаунтов и переводов используется Spring Data JPA с реализацией Hibernate. Этот инструмент автоматически генерирует SQL-запросы на основе методов репозитория и сокращает объём шаблонного кода при работе со связанными сущностями. В сервисе карт используется JDBC Template – низкоуровневый инструмент Spring для работы с базой данных через явно написанные SQL-запросы. Такой выбор обоснован простотой схемы данных сервиса карт (одна таблица) и стремлением к полному контролю над запросами без лишней абстракции.

HTML, CSS, JavaScript. Для фронтенда выбран стандартный веб-стек без использования фреймворков и библиотек. Взаимодействие с сервисами реализовано через встроенный браузерный Fetch API. Такой подход устраняет необходимость в системе сборки, снижает порог вхождения и делает код прозрачным. В отличие от React или Vue, чистый JavaScript не требует дополнительных инструментов для запуска и позволяет сосредоточиться на логике взаимодействия с API, а не на специфике фреймворка.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Физическая структура

Проект состоит из трёх независимых частей: сервис accounts, сервис cards и фронтенд. Каждый сервис представляет собой отдельное Java-приложение со своей структурой пакетов, конфигурацией и базой данных. Фронтенд – набор статических файлов, не связанных с системой сборки Java.

Структура сервиса accounts:

- config/ – конфигурационные классы;
- controller/ – REST-контроллеры;
- service/ – бизнес-логика;
- repository/ – интерфейсы Spring Data JPA;
- entity/ – JPA-сущности (таблицы БД);
- model/ – доменные модели;
- dto/ – объекты передачи данных (Request/Response);
- converter/ – конвертеры между слоями.

Структура сервиса cards:

- config/ – конфигурационные классы;
- controller/ – REST-контроллеры;
- service/ – бизнес-логика;
- repository/ – репозитории на основе JDBC Template;
- entity/ – классы, отражающие строки таблиц;
- model/ – доменные модели;
- dto/ – объекты передачи данных (Request/Response);
- converter/ – конвертеры между слоями.

Структура фронтенда:

- index.html – единственная HTML-страница приложения.

3.2 Описание разработанных модулей

Сервис accounts:

1 Пакет config содержит два конфигурационных класса. Первый настраивает CORS: разрешает запросы от фронтенда к REST-эндпоинтам сервиса, указывая допустимые источники, методы и заголовки. Без этой настройки браузер блокировал бы запросы фронтенда как cross-origin. Второй класс объявляет бин SecurityFilterChain, который обеспечивает доступ к эндпоинтам контроллеров только с помощью логина и пароля через HTTP Basic Authentication.

2 Пакет `entity` содержит JPA-сущности – Java-классы, каждый из которых соответствует таблице в базе данных. Сущность `AccountEntity` аннотирована `@Entity` и `@Table`, её поля отображаются на столбцы таблицы счетов: идентификатор, номер счёта, баланс, валюта и статус. Сущность `TransferEntity` аналогичным образом отображает таблицу переводов: идентификатор, идентификаторы счёта-источника и счёта-получателя, сумма, валюта и дата операции. Сущность `FavouriteTransfer` хранит идентификатор избранного перевода, номер счёта-отправителя, номер счёта-получателя и сумму перевода. Связи между сущностями объявлены через аннотации `@ManyToOne` и `@OneToMany` там, где это необходимо для навигации по графу объектов.

3 Пакет `model` содержит доменные модели – простые Java-классы, не зависящие от JPA и базы данных. Они используются внутри сервисного слоя для передачи данных между методами. Разделение на `entity` и `model` позволяет изолировать слой базы данных: изменения в схеме таблиц не затрагивают бизнес-логику напрямую.

4 Пакет `dto` содержит объекты передачи данных, разделённые на два типа для каждой сущности. Request-DTO принимают данные от фронтенда: например, `CreateAccountRequestDto` содержит поля, которые оператор заполняет при создании счёта, а `TransferRequestDto` – номера счетов и сумму перевода. Response-DTO формируют ответ, который сервис возвращает клиенту: например, `AccountResponseDto` включает все поля счёта, которые нужно отобразить в интерфейсе. Такое разделение даёт контроль над тем, какие поля принимаются на вход и какие возвращаются на выход, независимо от структуры `entity`.

5 Пакет `converter` содержит конвертеры, которые преобразуют объекты между слоями: из `entity` в `model`, из `model` в Response-DTO, из Request-DTO в `model`. Каждый конвертер – отдельный класс с методами преобразования. Вынесение этой логики в отдельный пакет упрощает тестирование и устраняет дублирование кода в сервисном слое.

6 Пакет `repository` содержит интерфейсы Spring Data JPA. Каждый интерфейс расширяет `JpaRepository` и автоматически получает стандартные методы: сохранение, поиск по идентификатору, получение всего списка, удаление. При необходимости в интерфейсах объявляются дополнительные методы. Например, поиск переводов по идентификатору счёта и диапазону дат для формирования выписки. Spring Data JPA генерирует реализацию этих методов автоматически на основе их имён или аннотации `@Query`.

7 Пакет `service` содержит сервисные классы, каждый из которых аннотирован `@Service` и инкапсулирует бизнес-логику для своей предметной области. `AccountService` реализует логику работы со счетами: создание нового счёта, получение списка всех счетов, получение счёта по номеру. При создании счёта сервис валидирует входные данные и вызывает репозиторий для сохранения сущности. `TransferService` реализует логику переводов. При проведении перевода сервис проверяет существование обоих счетов,

убеждается в достаточности баланса на счёте-источнике, списывает сумму с одного счёта, зачисляет на другой и сохраняет запись о переводе. Все три операции выполняются в рамках одной транзакции, аннотированной `@Transactional`, что гарантирует атомарность: если любой из шагов завершается с ошибкой, все изменения откатываются. Реализует логику избранных переводов: сохранение нового шаблона, получение списка всех шаблонов и удаление шаблона по идентификатору

8 Пакет `controller` содержит контроллеры, аннотированные `@RestController`. Каждый контроллер принимает HTTP-запросы, делегирует обработку соответствующему сервису и возвращает `Response-DTO` в формате JSON. `AccountController` обрабатывает запросы по маршруту `/accounts`. Метод создания счёта принимает POST-запрос с телом `CreateAccountRequestDto` и возвращает `AccountResponseDto` с кодом 201. Метод получения списка счетов обрабатывает GET-запрос и возвращает список `AccountResponseDto`. Метод получения выписки принимает номер счёта и временной диапазон как параметры запроса и возвращает отфильтрованный список переводов. `TransferController` обрабатывает запросы по маршруту `/transfers`. Метод проведения перевода принимает POST-запрос с телом `TransferRequest`. По маршруту `/transfers/favourite` размещены методы для работы с избранными переводами: POST для сохранения нового шаблона, GET для получения списка и DELETE для удаления по идентификатору.

9 Пакет `exception` содержит централизованный обработчик ошибок на основе `@ControllerAdvice`. Отдельный класс перехватывает исключения, которые выбрасываются в сервисном слое, и преобразует их в HTTP-ответы с понятными статусами и сообщениями. Например, если счёт с указанным идентификатором не найден, сервисный слой выбрасывает кастомное исключение `EntityNotFoundException`, а обработчик перехватывает его и возвращает ответ с кодом 404 и описанием ошибки в теле. Если баланс счёта недостаточен для перевода, возвращается код 400 с соответствующим сообщением. Такой подход устраняет дублирование логики обработки ошибок в каждом контроллере.

Сервис `cards`:

1 Пакет `config`, как и в сервисе `accounts`, содержит конфигурацию CORS. Также он содержит настройки клиента для запросов к сервису `accounts`. Клиент здесь используется активно: сервис `cards` не имеет доступа к базе данных `accounts`, поэтому для проверки существования счёта перед выпуском карты сервис отправляет HTTP-запрос к сервису `accounts` через настроенный клиент.

2 Пакет `entity` содержит один класс `CardEntity`, поля которого соответствуют столбцам таблицы карт: идентификатор карты, имя владельца, номер карты, номер привязанного счёта, статус и срок действия. В отличие от сервиса `accounts`, этот класс не является JPA-сущностью и не использует аннотаций `Hibernate`. Он служит простым Java-объектом для маппинга строк из результатов JDBC-запросов.

3 Пакет `model` содержит доменную модель `Card` – класс, используемый в сервисном слое. Отделение модели от `entity` позволяет менять детали хранения данных, не затрагивая бизнес-логику.

4 Пакет `dto` содержит `CreateCardRequestDto` для приёма данных при выпуске карты (номер счёта, имя владельца), `CardResponseDto` для возврата данных о карте клиенту и `CardChangeStatusRequestDto` для изменения статуса карты. Разделение на `Request` и `Response` аналогично подходу в сервисе `accounts`.

5 Пакет `converter` содержит конвертер, преобразующий `CardEntity` и `Card`, `Card` в `CardResponseDto`, `CreateCardRequestDto` в `Card`. Логика маппинга сосредоточена в одном месте и не дублируется в сервисном слое.

6 Пакет `repository` содержит класс `CardRepositoryImpl`, аннотированный `@Repository`. В отличие от интерфейсов `Spring Data JPA` в сервисе `accounts`, этот класс написан вручную и использует `JdbcTemplate` для выполнения SQL-запросов напрямую. Каждый метод репозитория содержит явный SQL-запрос: получение всех карт, сохранение новой карты, обновление статуса карты, удаление карты по идентификатору. `JdbcTemplate` берёт на себя открытие соединения, передачу параметров и закрытие ресурсов, освобождая код от шаблонного управления `JDBC`.

7 Пакет `service` содержит `CardServiceImpl`, аннотированный `@Service`. Сервис реализует четыре операции: выпуск карты, получение списка карт, изменение статуса карты (блокировка и разблокировка) и удаление карты. Также пакет содержит `AccountServiceImpl`, нужный для проверки наличия счёта при создании карты. При выпуске карты сервис `CardServiceImpl` сначала вызывает сервис `accounts` через `AccountServiceImpl`, чтобы убедиться в существовании указанного счёта. Если счёт не найден, выбрасывается исключение и карта не создаётся. При успешной проверке сервис генерирует номер карты, устанавливает начальный статус и сохраняет карту через репозиторий. При блокировке и разблокировке сервис получает карту по идентификатору, проверяет текущий статус и обновляет его.

8 Пакет `controller` содержит `CardController`, который аннотирован `@RestController` и обрабатывает запросы по маршруту `/cards`. Метод выпуска карты принимает POST-запрос с телом `CreateCardRequestDto` и возвращает `CardResponseDto` с кодом 201. Метод получения списка карт возвращает список `CardResponseDto`. Методы блокировки и разблокировки принимают PATCH-запросы с идентификатором карты в пути. Метод удаления обрабатывает DELETE-запрос и возвращает код 204 без тела ответа.

9 Пакет `exception` содержит обработку ошибок, которая реализована через `@ControllerAdvice` аналогично сервису `accounts`: кастомные исключения перехватываются и преобразуются в HTTP-ответы с соответствующими статусами.

Фронтенд представляет собой одностраничное приложение (SPA) на основе единственного файла `index.html`. Вся логика навигации и отображения

данных реализована на чистом JavaScript без использования фреймворков. Страница разделена на вкладки, каждая из которых соответствует одному разделу интерфейса: счета, переводы, избранные переводы, карты. Переключение между вкладками реализовано через навигационное меню: при клике на пункт меню JavaScript показывает нужную секцию и скрывает остальные, одновременно обновляя заголовок страницы и при необходимости иницилируя загрузку соответствующих данных с сервера. Такой подход обеспечивает быструю работу интерфейса без перезагрузок, а также позволяет сохранять контекст между разделами – например, оператор может открыть несколько счетов в разных вкладках браузера или вернуться к заполненной форме перевода после проверки баланса в другом разделе.

Взаимодействие с бэкендом реализовано через встроенный браузерный Fetch API. Каждое действие оператора (загрузка списка счетов, создание перевода, выпуск карты) иницирует асинхронный HTTP-запрос к соответствующему сервису. Запросы к сервису accounts (управление счетами и переводами) направляются на один адрес и порт (по умолчанию localhost:8081) с использованием базовой HTTP-аутентификации, запросы к сервису cards – на другой адрес и порт (localhost:8080) без аутентификации. Адреса сервисов заданы как константы в JavaScript-файле, но оператор может динамически изменять их через текстовые поля в боковой панели без перезагрузки страницы – это полезно для переключения между тестовой, staging и production средами. Для унификации всех вызовов используется общая функция-обёртка apiFetch(), которая автоматически подставляет нужный базовый URL, заголовки и обрабатывает некорректные ответы.

После получения ответа JavaScript разбирает JSON и динамически обновляет DOM: заполняет таблицы строками, показывает или скрывает элементы, выводит сообщения об ошибках. При возникновении ошибки на стороне сервера (коды 4xx и 5xx) фронтенд извлекает сообщение из тела ответа и отображает его оператору без перезагрузки страницы. Для этого используется единый механизм обработки ошибок в apiFetch(): при любом неуспешном статусе ответа функция выбрасывает исключение, которое затем перехватывается в конкретной операции, после чего ошибка отображается как в виде всплывающего уведомления (toast), так и внутри соответствующего блока интерфейса – например, прямо в таблице вместо загруженных данных. Пользовательские данные перед вставкой в HTML проходят через функцию экранирования спецсимволов, что предотвращает XSS-атаки. Дополнительно реализована клиентская валидация (проверка длины IBAN, обязательность полей, положительность суммы), которая уменьшает количество бесполезных запросов к серверу и даёт оператору мгновенную обратную связь.

4 ТЕСТИРОВАНИЕ

Модульное тестирование направлено на проверку отдельных классов в полной изоляции от внешних зависимостей – базы данных, сети и других сервисов. Все модульные тесты написаны для сервиса accounts с использованием JUnit 5, Mockito и AssertJ.

Зависимости каждого тестируемого класса заменяются моками с помощью аннотации `@Mock`. Там где нужно частичное поведение реального объекта – например, для конвертеров – используется `@Spy`: такой объект выполняет настоящие методы, но позволяет при необходимости переопределить отдельные из них.

Тесты сгруппированы во вложенные классы с помощью аннотации `@Nested` – по одному вложенному классу на каждую группу сценариев одного метода. Проверка результатов выполняется через AssertJ: для простых значений используется `assertThat(...).isEqualTo(...)`, для объектов – `usingRecursiveComparison()`, которое сравнивает все поля рекурсивно. Проверка исключений выполняется через `assertThatThrownBy(...)`.

Тест `AccountServiceImplTes` тестирует класс `AccountServiceImpl`. Тесты покрывают три публичных метода: `create()`, `getAll()` и `getByIban()`.

Группа тестов `CreateTests` проверяет корректность инициализации счёта при создании. Первый сценарий убеждается, что сервис автоматически проставляет статус `ACTIVE` и генерирует непустой IBAN. Второй и третий сценарии проверяют обработку поля `settings`: если оно равно `null`, сервис создаёт объект настроек по умолчанию с выключенными SMS-уведомлениями; если объект настроек передан, но поле `smsNotificationsEnabled` не заполнено – сервис явно устанавливает его в `false`, не оставляя значение неопределённым. Четвёртый сценарий проверяет, что полностью заполненные настройки сохраняются без изменений: сервис не должен перезаписывать явно переданные значения. Пятый сценарий проверяет уникальность IBAN: при двух последовательных вызовах `create()` генерируемые идентификаторы должны различаться, что обеспечивает отсутствие коллизий.

Группа тестов `GetAllTests` проверяет два сценария. В первом репозиторий возвращает непустой список сущностей; тест убеждается, что сервис корректно конвертирует их в список доменных моделей и возвращает все элементы без потерь, используя `usingRecursiveComparison()` для поэлементного сравнения. Во втором сценарии репозиторий возвращает пустой список; ожидается, что сервис также вернёт пустой список, а не `null` и не выброшенное исключение.

Группа тестов `GetByIbanTests` проверяет два случая. В позитивном сценарии репозиторий возвращает сущность по заданному IBAN; тест проверяет, что возвращённая доменная модель содержит корректные данные. В негативном сценарии репозиторий возвращает пустой `Optional`; ожидается

выброс `EntityNotFoundException` с сообщением, идентифицирующим несуществующий IBAN.

Тест `CurrencyRateServiceImplTest` тестирует класс `CurrencyRateServiceImpl`, отвечающий за хранение и получение курсов валют. Тесты покрывают два метода: `getTodayRate()` и `saveAll()`.

Группа тестов `GetTodayRateTests` включает три сценария. Первый проверяет успешный случай: репозиторий находит запись для заданной валюты на текущую дату и возвращает её; тест сравнивает возвращённую модель с ожидаемой. Второй сценарий проверяет отсутствие курса: если репозиторий не находит запись – выбрасывается `EntityNotFoundException`. Третий сценарий параметризован по набору валют (USD, EUR, RUB) и убеждается, что сервис корректно обрабатывает каждую из них без специальной логики для конкретных кодов.

Группа тестов `SaveAllTests` проверяет три случая. Первый убеждается, что список курсов передаётся в репозиторий без изменений и вызов `saveAll()` выполняется ровно один раз. Второй проверяет, что пустой список обрабатывается без ошибок: метод завершается нормально и не вызывает исключений. Третий сценарий передаёт большой список (несколько сотен записей) и проверяет, что метод завершается корректно, – это защищает от ошибок при пакетной обработке данных.

Тест `NationalBankServiceTest` тестирует класс `NationalBankService`, который отвечает за получение курсов валют из внешнего HTTP-API Национального банка и их преобразование в доменные модели. Тестируется метод `fetchDailyRates()`.

Первый сценарий проверяет фильтрацию: API Национального банка возвращает курсы для всех торгуемых валют, однако система поддерживает только USD, EUR и RUB. Тест передаёт mock-ответ со смешанным набором валют и убеждается, что в результирующем списке присутствуют только поддерживаемые коды, а остальные отброшены. Второй сценарий проверяет корректность конвертации DTO в доменную модель `CurrencyRate`: все поля – код валюты, курс, масштаб и дата – должны перенестись без потерь и искажений. Третий сценарий передаёт mock-ответ, содержащий исключительно неподдерживаемые валюты, и ожидает возврат пустого списка без ошибок. Четвёртый сценарий имитирует пустой ответ API и также ожидает пустой список на выходе.

Тест `TransferServiceImplTest` тестирует класс `TransferServiceImpl`. Это наиболее обширный набор тестов: он охватывает математику конвертации валют, граничные случаи и все виды ошибок.

Позитивные сценарии проверяют переводы в одинаковой валюте: для пары USD-USD и BYN-BYN конвертация не применяется, сумма списывается с отправителя и зачисляется получателю напрямую. Тест проверяет итоговые балансы обоих счетов и убеждается, что запись о переводе сохранена с корректными данными.

Сценарии с конвертацией проверяют три комбинации валют. При переводе USD-EUR сумма сначала конвертируется из USD в BYN по курсу USD/BYN, затем из BYN в EUR по курсу EUR/BYN – применяются два курса последовательно. При переводе BYN-EUR применяется один курс. При переводе USD-BYN также применяется один курс. В каждом сценарии тест проверяет математическую точность результата: итоговые балансы сравниваются с ожидаемыми значениями с точностью до двух знаков после запятой. Отдельно проверяется корректная работа с масштабом валюты: для RUB, где масштаб равен 100, сервис должен делить курс на масштаб перед применением, иначе результат будет в 100 раз меньше ожидаемого.

Негативные сценарии охватывают пять видов ошибок. Перевод с неактивного счёта-отправителя или на неактивный счёт-получатель завершается выбросом `IllegalArgumentException` – статус проверяется до любых финансовых операций. При недостаточном балансе на счёте-отправителе выбрасывается соответствующее исключение, а состояние счетов остаётся неизменным. При совпадении IBAN отправителя и получателя операция отклоняется до обращения к базе данных. Если любой из счетов не существует – выбрасывается `EntityNotFoundException` с указанием несуществующего идентификатора.

Интеграционное тестирование проверяет взаимодействие нескольких компонентов системы совместно: реальную базу данных, HTTP-клиенты, планировщики задач и REST-эндпоинты. В отличие от модульных тестов, здесь не используются моки для инфраструктурных зависимостей – компоненты работают так же, как в рабочем окружении.

Все интеграционные тесты используют `Testcontainers`: библиотека запускает изолированный экземпляр PostgreSQL в Docker-контейнере на время выполнения тестов. Это гарантирует, что тесты работают с реальной СУБД, а не с упрощённой in-memory заменой, и исключает влияние на рабочую или разработческую базу данных. Перед каждым тестом база данных очищается в методе, аннотированном `@BeforeEach`, через вызов `cleanDatabase()` или `deleteAll()` – это обеспечивает полную независимость тестов друг от друга вне зависимости от порядка их выполнения.

Тест `FetchAndSaveDailyCurrencyRatesIT` проверяет полный цикл получения и сохранения курсов валют: от HTTP-запроса к внешнему API до записи результатов в базу данных PostgreSQL. Тестируемые компоненты – `CurrencyRateScheduler`, `NationalBankClient` и репозиторий курсов – поднимаются вместе в реальном Spring-контексте.

Внешний API Национального банка имитируется с помощью `WireMockExtension`. `WireMock` запускает локальный HTTP-сервер, который перехватывает исходящие запросы от `NationalBankClient` и возвращает заранее подготовленные ответы. Это позволяет полностью контролировать поведение внешнего сервиса в тестах: задавать конкретные данные, имитировать ошибки и проверять, что приложение корректно реагирует на каждый из вариантов.

Позитивные сценарии проверяют успешный путь: WireMock возвращает корректный JSON с курсами нескольких валют, планировщик запускается и вызывает клиент, клиент получает ответ и передаёт его в сервис, сервис фильтрует и сохраняет данные. После завершения тест обращается к базе данных напрямую и проверяет три вещи: сохранены только поддерживаемые валюты (EUR, USD, RUB), количество записей соответствует ожидаемому, а значения курса, масштаба и даты для каждой валюты совпадают с данными из mock-ответа.

Негативные сценарии проверяют три вида сбоев. В первом WireMock возвращает HTTP 500 – сервис должен обработать ошибку без выброса непойманного исключения, а база данных должна остаться пустой. Во втором WireMock возвращает пустой JSON-массив – результат тот же: ни одной записи в базе. В третьем mock-ответ содержит только неподдерживаемые валюты – фильтрация должна отбросить все записи, и база данных также остаётся пустой. Все три сценария дополнительно проверяют, что планировщик не выбрасывает исключение при любом из этих исходов: сбой внешнего API не должен приводить к падению приложения.

Тест TransferApiTest проверяет REST API переводов на уровне HTTP: реальные запросы к запущенному Spring-приложению, реальная база данных в Testcontainers. Для доступа к эндпоинту POST /transfers используется базовая HTTP-аутентификация с учётными данными user/user123. Перед каждым тестом в базу предзагружаются курсы валют, необходимые для конвертации, а также тестовые счета с заданными балансами.

Позитивные сценарии проверяют два вида успешного перевода. Перевод без конвертации (USD→USD): запрос отправляется с указанием суммы, IBAN отправителя и получателя; тест получает код 200 в ответе, затем считывает балансы обоих счетов из базы и сравнивает их с ожидаемыми – у отправителя должна уменьшиться сумма перевода, у получателя увеличиться на ту же величину. Перевод с конвертацией (USD→BYN): аналогичная проверка, но ожидаемый баланс получателя рассчитывается с учётом курса, предзагруженного в базу перед тестом.

Негативные сценарии охватывают шесть видов ошибок на уровне API. При недостаточном балансе на счёте-отправителе сервис возвращает код 400; тест дополнительно проверяет, что балансы обоих счетов в базе не изменились. При отсутствии счёта-отправителя или счёта-получателя возвращается код 404 с описанием ошибки. При совпадении IBAN отправителя и получателя – код 400. При передаче IBAN в некорректном формате – код 400, возникающий ещё на этапе валидации входных данных до обращения к базе. При попытке перевода с заблокированного счёта или на заблокированный счёт – код 400. Во всех негативных сценариях тест проверяет не только статус ответа, но и неизменность состояния базы данных: отклонённая операция не должна частично изменять балансы или создавать запись о переводе.

5 ПРИМЕНЕНИЕ ПРОГРАММЫ

5.1 Руководство пользователя

Веб-приложение BankAdmin предназначено для операторов банка и предоставляет централизованный интерфейс управления счетами, переводами и платёжными картами. Приложение открывается в браузере и не требует установки.

Интерфейс разделён на три зоны. Слева расположена боковая панель навигации, через которую оператор переключается между разделами приложения. В верхней части страницы находится заголовок, отображающий название текущего раздела. Основная часть экрана занята рабочей областью, содержимое которой меняется в зависимости от выбранного раздела. Переключение между разделами происходит без перезагрузки страницы.

Навигационная панель разделена на три группы: «Счета», «Переводы» и «Карты». Каждая группа содержит соответствующие пункты меню. Активный пункт выделяется цветом.

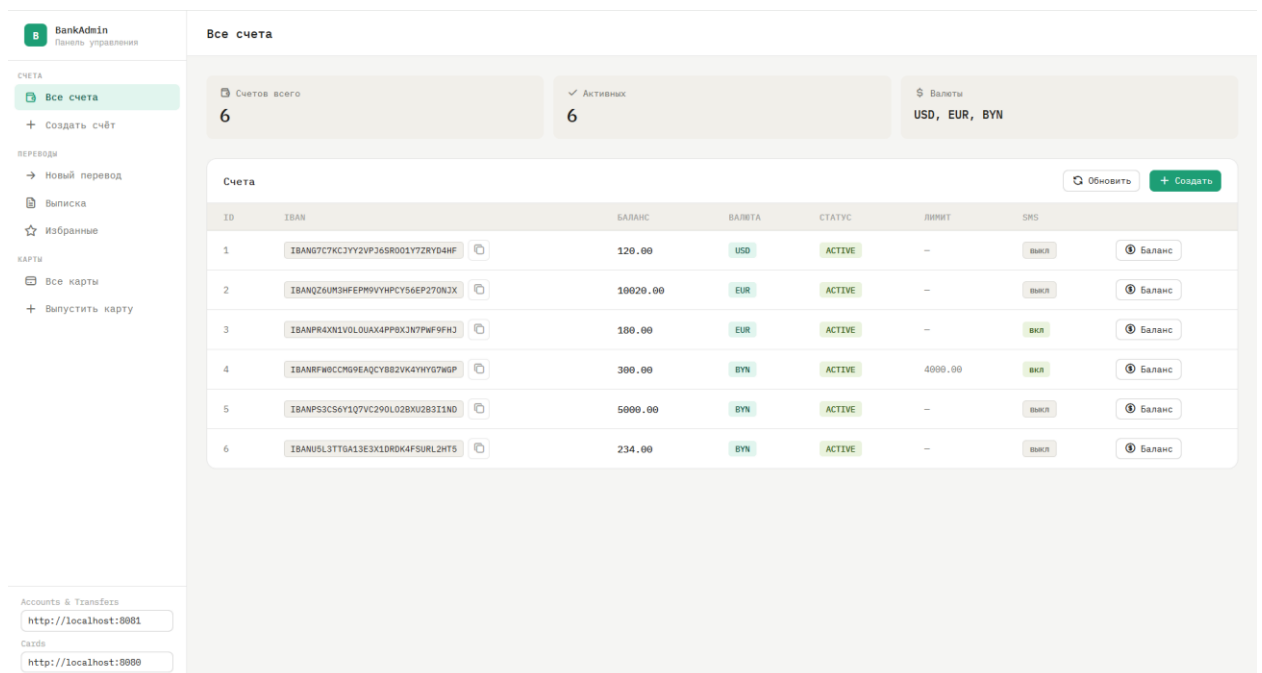


Рисунок 5.1 – Общий вид интерфейса с боковой панелью

При открытии приложения оператор попадает в раздел «Все счета». Здесь отображается таблица со списком всех банковских счетов. Для каждого счёта в таблице показаны его номер (IBAN), имя владельца, баланс, валюта и текущий статус. Активные счета отмечены зелёным значком, закрытые – серым.

В правом верхнем углу таблицы расположена кнопка «Создать счёт», которая переводит оператора на страницу создания нового счёта.

Счета								Обновить	Создать
ID	IBAN	Баланс	Валюта	Статус	Лимит	SMS			
1	IBAN67C7KCJYY2VP36SR001Y7ZRYD4HF	120.00	USD	ACTIVE	—	выкл	Баланс		
2	IBANQZ6UM3HFEPW9YHPCY56EP270NJX	10020.00	EUR	ACTIVE	—	выкл	Баланс		
3	IBANPR4XN1VOLUUA4P0XJN7PMF9FHJ	100.00	EUR	ACTIVE	—	вкл	Баланс		
4	IBANRFW8CCMG9EAQCY882VK4YHYG7WGP	300.00	BYN	ACTIVE	4000.00	вкл	Баланс		
5	IBANPS3CS6Y1Q7VC290L02BXU2B3I1ND	5000.00	BYN	ACTIVE	—	выкл	Баланс		
6	IBANU5L3TTGA13E3X1DRDK4FSURL2HT5	234.00	BYN	ACTIVE	—	выкл	Баланс		

Рисунок 5.2 – Список счетов

Форма создания счёта содержит следующие поля: начальный баланс, валюта (выбирается из списка), необязательный месячный лимит и переключатель SMS-уведомлений. Поля, отмеченные звёздочкой, обязательны для заполнения.

После заполнения формы оператор нажимает кнопку подтверждения. Сервис автоматически генерирует IBAN и присваивает счёту статус «Активен». Кнопка «Отмена» возвращает оператора к списку счетов без сохранения данных.

BankAdmin

Панель управления

СЧЕТА

Все счета

Создать счёт

ПЕРЕВОДЫ

Новый перевод

Выписка

Избранные

КАРТЫ

Все карты

Выпустить карту

Accounts & Transfers

http://localhost:8081

Cards

http://localhost:8080

Создать счёт

Начальный баланс *

0.00

Валюта *

— выбрать —

Настройки (необязательно)

Месячный лимит

без лимита

SMS-уведомления

Выключено

Отмена

Создать счёт

Рисунок 5.3 – Форма создания счёта

Форма перевода содержит три обязательных поля: IBAN счёта-отправителя (32 символа), IBAN счёта-получателя (32 символа) и сумма перевода. Валюта определяется автоматически по валюте счёта-отправителя; если валюта счетов различается, система выполняет конвертацию по актуальному курсу.

После заполнения полей оператор подтверждает операцию. Если данные корректны и баланс достаточен – перевод проводится, балансы обоих счетов обновляются. В случае ошибки (недостаточный баланс, несуществующий счёт, заблокированный счёт) под формой появляется сообщение с описанием причины отказа.

The screenshot displays the 'BankAdmin' web application interface. The main heading is 'Новый перевод' (New Transfer). The form contains three input fields: 'IBAN отправителя * (32 символа)' (IBAN of the sender, 32 symbols), 'IBAN получателя * (32 символа)' (IBAN of the recipient, 32 symbols), and 'Сумма *' (Amount). The 'Сумма' field is pre-filled with '0.00'. Below the form are two buttons: 'Отмена' (Cancel) and 'Выполнить перевод' (Execute Transfer). The left sidebar has a menu with 'СЧЕТА' (Accounts), 'ПЕРЕВОДЫ' (Transfers), and 'КАРТЫ' (Cards). The 'ПЕРЕВОДЫ' section is active, showing 'Новый перевод' as the selected option. At the bottom, there are two links: 'http://localhost:8001' and 'http://localhost:8000'.

Рисунок 5.4 – Форма создания перевода

В левой части экрана расположена панель «Избранные переводы». Она позволяет быстро заполнить форму реквизитами сохранённого получателя – достаточно выбрать нужный шаблон из списка.

☆ Избранный перевод

IBAN отправителя * (32)

0/32

IBAN получателя * (32)

0/32

Сумма *

0.00

Отмена Сохранить ☆

Рисунок 5.5 – форма создания избранного перевода

Раздел «Выписка» предназначен для просмотра истории операций по счёту. Оператор вводит IBAN счёта, дату начала и дату конца периода, после чего загружается таблица переводов. Для каждой операции отображаются IBAN отправителя, IBAN получателя, сумма, валюта и дата проведения.

Заголовок таблицы показывает количество найденных операций за выбранный период.

BankAdmin

Выписка

Выписка по счёту

IBAN счёта *

IBANQZ6UM3HFEP9YUHCY56EP270M3X

Начало периода *

01.06.2026

Конец периода *

16.06.2026

Получить выписку

Операции (9)

Дата	Время	Откуда	Куда	Сумма	Валюта
2026-06-08	15:25	IBANPR4XN1VOLO..	IBANQZ6UM3HFEP..	20.00	EUR
2026-06-08	15:25	IBANPR4XN1VOLO..	IBANQZ6UM3HFEP..	20.00	EUR
2026-06-08	15:25	IBANPR4XN1VOLO..	IBANQZ6UM3HFEP..	20.00	EUR
2026-06-08	15:25	IBANPR4XN1VOLO..	IBANQZ6UM3HFEP..	20.00	EUR
2026-06-08	15:28	IBANQZ6UM3HFEP..	IBANPR4XN1VOLO..	60.00	EUR
2026-06-08	15:28	IBANQZ6UM3HFEP..	IBANPR4XN1VOLO..	60.00	EUR
2026-06-08	15:19	IBANPR4XN1VOLO..	IBANQZ6UM3HFEP..	20.00	EUR
2026-06-08	14:24	IBANPR4XN1VOLO..	IBANQZ6UM3HFEP..	20.00	EUR
2026-06-08	12:44	IBANPR4XN1VOLO..	IBANQZ6UM3HFEP..	20.00	EUR

Accounts & Transfers

http://localhost:8081

Cards

http://localhost:8080

Рисунок 5.6 – Выписка по счёту

Раздел «Избранные» отображает все сохранённые шаблоны переводов в виде таблицы. Для каждого шаблона показаны IBAN счёта-отправителя и IBAN счёта-получателя.

Новый шаблон добавляется через боковую панель, которая открывается кнопкой на странице нового перевода. Оператор вводит IBAN отправителя, IBAN получателя и сумму, после чего шаблон сохраняется и становится доступен в списке. Для удаления шаблона оператор нажимает кнопку удаления в соответствующей строке таблицы.

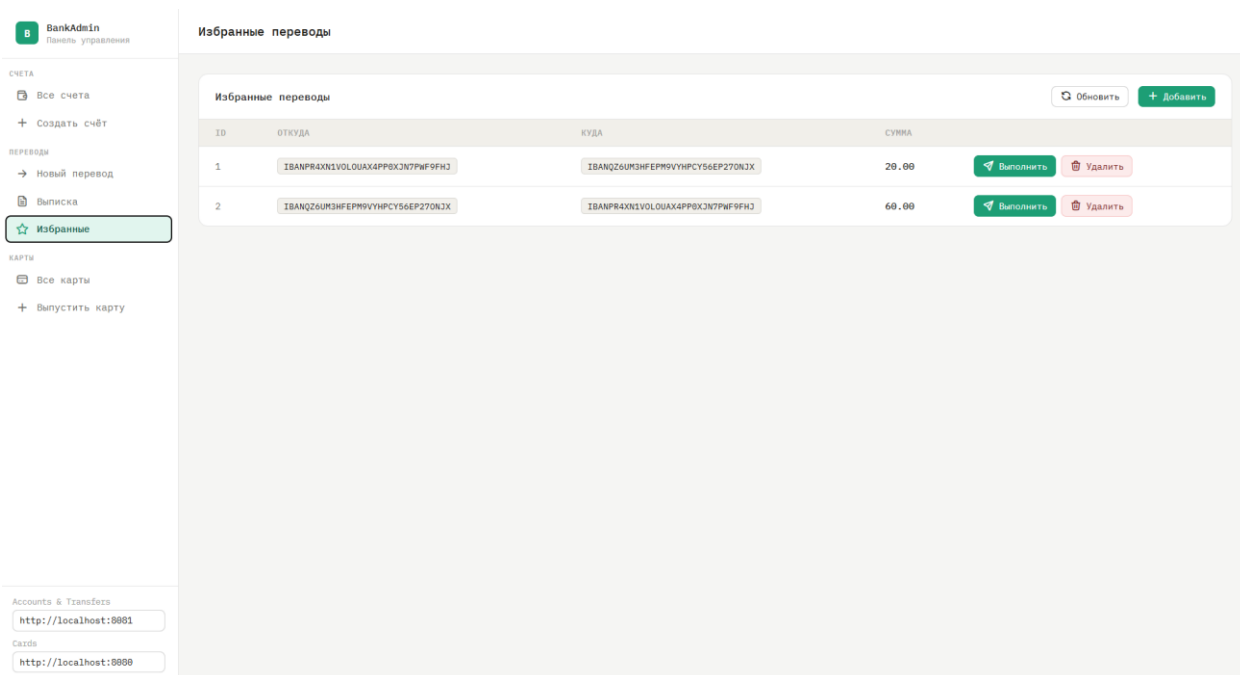


Рисунок 5.7 – Список избранных переводов

Раздел «Все карты» отображает таблицу всех платёжных карт. Для каждой карты указаны её номер, IBAN привязанного счёта, имя держателя, срок действия и статус (активна или заблокирована).

Для изменения статуса карты оператор нажимает кнопку действия в строке нужной карты. Открывается модальное окно, в котором отображается текущий статус карты и предлагается выбрать новый. После подтверждения статус обновляется. Для удаления карты оператор нажимает соответствующую кнопку – карта удаляется без возможности восстановления.

В правом верхнем углу таблицы расположена кнопка «Выпустить карту», переводящая в раздел создания карты.

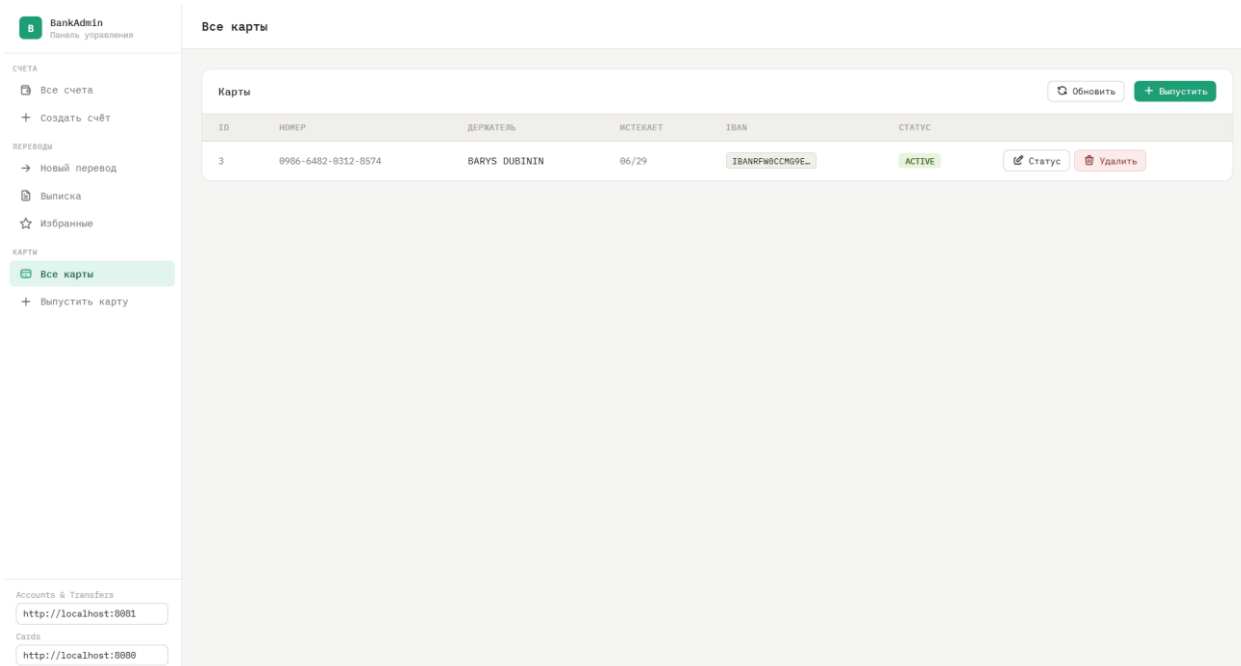


Рисунок 5.8 – Список карт

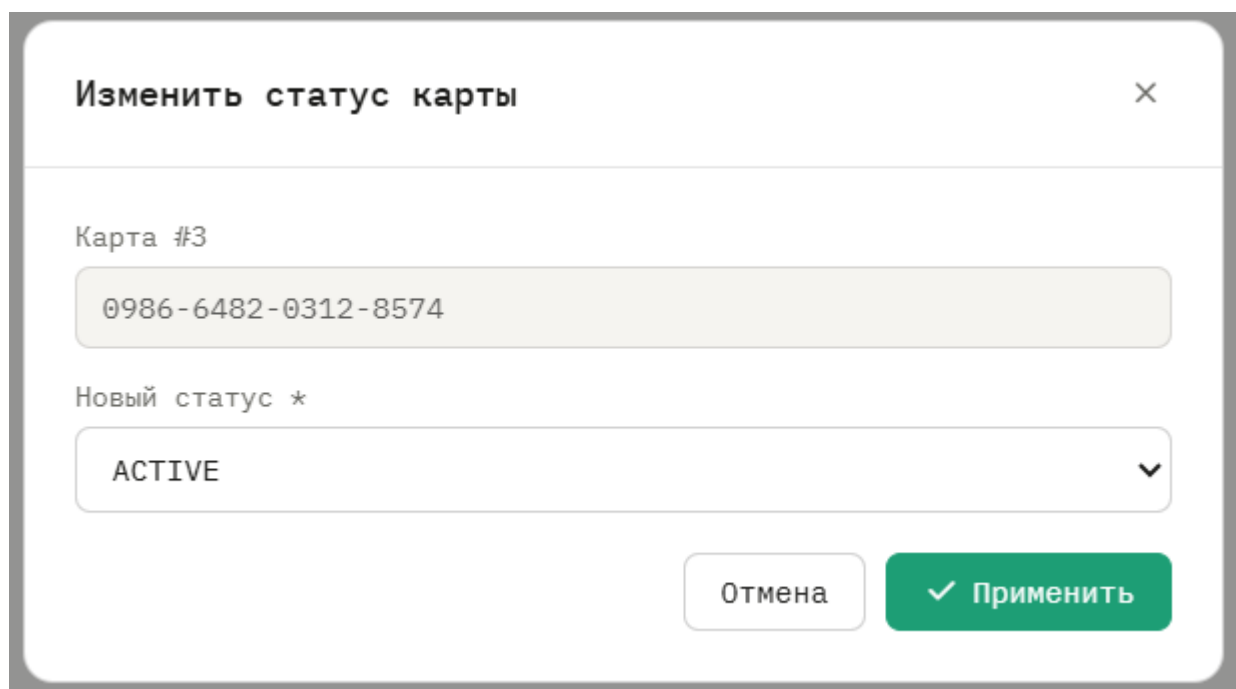


Рисунок 5.9 – Модальное окно изменения статуса

Форма выпуска карты содержит два обязательных поля: имя держателя карты и IBAN счёта, к которому привязывается карта (32 символа). После заполнения и подтверждения сервис проверяет существование указанного счёта. Если счёт найден – карта создаётся и появляется в общем списке карт. Если счёт не найден – выводится сообщение об ошибке. Кнопка «Отмена» возвращает оператора к списку карт.

BankAdmin
Панель управления

Выпустить карту

СЧЕТА

- Все счета
- + Создать счёт

ПЕРЕВОДЫ

- Новый перевод
- Выписка
- Избранные

КАРТЫ

- Все карты
- + Выпустить карту

Accounts & Transfers
<http://localhost:8081>

Cards
<http://localhost:8080>

Выпустить карту

Имя держателя *

USER USEROVICH

IBAN счёта * (32 символа)

IBANG7C7KCJY2VP36SR001Y7ZRYD4HF

32/32

Отмена

Рисунок 5.10 – Форма выпуска карты

BankAdmin
Панель управления

Выпустить карту

IBAN должен быть ровно 32 символа

Выпустить карту

Имя держателя *

USER USEROVICH

IBAN счёта * (32 символа)

IBANG7C7KCJY2VP36SR001

23/32

Отмена

Рисунок 5.11 – Неправильно заполненная форма выпуска карты

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано веб-приложение для управления банковскими счетами и картами. Приложение реализует централизованный административный интерфейс, через который оператор выполняет полный набор банковских операций: создание и просмотр счетов, проведение переводов с автоматической конвертацией валют по курсам Национального банка, формирование выписок за произвольный период, управление избранными переводами, а также выпуск, блокировка и удаление платёжных карт. Все операции выполняются через единый интерфейс без необходимости переключения между несколькими системами, что снижает нагрузку на оператора и уменьшает вероятность ошибок при ручной обработке данных.

В процессе разработки были решены следующие задачи: проведён анализ предметной области и существующих решений; спроектирована архитектура приложения на основе двух независимых микросервисов; реализован бэкенд с использованием Spring Boot и Spring Framework; разработан фронтенд в виде одностраничного приложения на чистом HTML, CSS и JavaScript; написаны модульные и интеграционные тесты, покрывающие основные сценарии работы системы. Разделение бэкенда на два сервиса – сервис аккаунтов и переводов и сервис карт – обеспечивает независимость их разработки и развёртывания, а взаимодействие между ними по протоколу HTTP делает архитектуру прозрачной и легко расширяемой.

Разработанное приложение демонстрирует практическое применение современных инструментов серверной разработки на Java: использование Spring Data JPA для работы с реляционными данными, JDBC Template для низкоуровневого управления запросами, REST API для межсервисного взаимодействия и синхронных вызовов через RestTemplate. Тестирование с применением JUnit 5, Mockito, AssertJ, Testcontainers и WireMock позволило проверить корректность бизнес-логики и устойчивость системы к ошибочным входным данным. Особое внимание в тестировании уделено математической точности конвертации валют и корректной обработке граничных случаев.

К недостаткам текущей реализации можно отнести отсутствие аутентификации на уровне фронтенда, ограниченную валидацию вводимых данных на стороне клиента, а также отсутствие пагинации в таблицах при большом количестве записей.

В качестве направлений дальнейшего развития можно выделить добавление ролевой модели доступа, реализацию журнала аудита операций, поддержку уведомлений в реальном времени, а также переход к асинхронному межсервисному взаимодействию через очередь сообщений. Внедрение этих улучшений позволило бы приблизить приложение к уровню производственных систем и расширить круг решаемых им задач.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода с комментариями

AccountController.java:

```
package org.example.controller;
```

```
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.ArraySchema;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.annotation.LogExecutionTime;
import org.example.converter.AccountModelAndDtoConverter;
import org.example.dto.*;
import org.example.model.Account;
import org.example.service.AccountService;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@Tag(name = "Accounts management", description = "API for accounts management")
```

```
@RestController
```

```
@RequestMapping("/accounts")
```

```
@RequiredArgsConstructor
```

```
public class AccountController {
```

```
    private final AccountService accountService;
```

```
    private final AccountModelAndDtoConverter accountConverter;
```

```
    @Operation(
```

```
        summary = "Create new account",
```

```
        description = "Creates an account with the specified details",
```

```
        responses = {
```

```
            @ApiResponse(responseCode = "201", description = "Account was successfully created",
```

```
                content = @Content(schema = @Schema(implementation = AccountResponseDto.class))),
```

```
            @ApiResponse(responseCode = "400", description = "Bad request")
```

```
        })
```

```
    @PostMapping
```

```
    @ResponseStatus(HttpStatus.CREATED)
```

```
    public AccountResponseDto create(@Valid @RequestBody CreateAccountRequestDto
```

```
createAccountRequestDto) {
```

```
        Account account = accountConverter.toModel(createAccountRequestDto);
```

```
        Account newAccount = accountService.create(account);
```

```
        return accountConverter.toDto(newAccount);
```

```
    }
```

```
    @Operation(
```

```
        summary = "Get all accounts",
```

```
        description = "Returns list of all accounts",
```

```
        responses = {
```

```
            @ApiResponse(responseCode = "200", description = "Accounts retrieved successfully", content =
```

```
                @Content(array = @ArraySchema(schema = @Schema(implementation =
```

```
AccountResponseDto.class)))
```

```
        )
```

```
        })
```

```
    @GetMapping
```

```

public List<AccountResponseDto> getAll() {
    List<Account> allAccounts = accountService.getAll();
    return accountConverter.toDtos(allAccounts);
}

@Operation(
    summary = "Get balance",
    description = "Returns balance and currency of the account",
    responses = {
        @ApiResponse(
            responseCode = "200", description = "Balance was successfully retrieved",
            content = @Content(schema = @Schema(implementation = GetBalanceResponseDto.class))),
        @ApiResponse(responseCode = "400", description = "Bad request"),
        @ApiResponse(responseCode = "404", description =
            "Specified resource didn't found (example: non-existent IBAN of the account)")
    })
@LogExecutionTime
@GetMapping("/balance")
public GetBalanceResponseDto getBalance(@RequestParam("iban") String iban) {
    Account account = accountService.getByIban(iban);
    return new GetBalanceResponseDto(account.getBalance(), account.getCurrency());
}
}

```

CreateAccountRequestDto.java:

```

package org.example.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PositiveOrZero;
import lombok.Getter;
import lombok.Setter;
import org.example.model.AccountCurrency;

import java.math.BigDecimal;

@Getter
@Setter
@Schema(description = "Request to create a new account")
public class CreateAccountRequestDto {

    @Schema(
        description = "Initial account balance",
        example = "1234.00",
        requiredMode = Schema.RequiredMode.REQUIRED
    )
    @NotNull(message = "Initial account balance is required")
    @PositiveOrZero(message = "Balance cannot be less than 0")
    private BigDecimal balance;

    @Schema(
        description = "Account currency (ISO 4217 code)",
        allowableValues = { "BYN", "RUB", "USD", "EUR" },
        requiredMode = Schema.RequiredMode.REQUIRED
    )

```

```

        @NotNull(message = "Account currency is required")
        private AccountCurrency currency;

        @Schema(
            description = "Account settings",
            requiredMode = Schema.RequiredMode.NOT_REQUIRED
        )
        private AccountSettingsDto settings;
    }

```

AccountResponseDto.java:

```

package org.example.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Getter;
import lombok.Setter;
import org.example.model.AccountStatus;
import org.example.model.AccountCurrency;

import java.math.BigDecimal;

@Setter
@Getter
@Schema(description = "The result of a successful request containing the account data")
public class AccountResponseDto {

    @Schema(description = "Unique account identifier", example = "1001")
    private Long id;

    @Schema(description = "Balance of the account", example = "1234.00")
    private BigDecimal balance;

    @Schema(description = "Account currency (ISO 4217 code)", allowableValues = {"RUB", "BYN", "USD", "EUR"})
    private AccountCurrency currency;

    @Schema(description = "Current status of the account")
    private AccountStatus status;

    @Schema(description = "IBAN of the account")
    private String iban;

    @Schema(description = "Account settings")
    private AccountSettingsDto settings;
}

```

TransferController.java:

```

package org.example.controller;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.converter.FavoriteTransferModelAndDtoConverter;
import org.example.converter.TransferModelAndDtoConverter;
import org.example.dto.*;

```

```

import org.example.model.FavoriteTransfer;
import org.example.model.Transfer;
import org.example.service.TransferService;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@Tag(name = "Transfers management", description = "API for transfers management")
@RestController
@RequestMapping("/transfers")
@RequiredArgsConstructor
public class TransferController {

    private final TransferModelAndDtoConverter transferConverter;
    private final FavoriteTransferModelAndDtoConverter favoriteTransferConverter;
    private final TransferService transferService;

    @Operation(
        summary = "Perform the transfer",
        description = "Performs the transfer between specified accounts with specified amount",
        responses = {
            @ApiResponse(
                responseCode = "200", description = "Transfer was successfully performed",
                content = @Content(schema = @Schema(implementation = TransferResponseDto.class))),
            @ApiResponse(responseCode = "400", description = "Bad request"),
            @ApiResponse(responseCode = "404", description =
                "Specified resource didn't found (example: non-existent IBAN of the account)")
        })
    @PostMapping
    public TransferResponseDto performTransfer(@Valid @RequestBody TransferRequestDto transferRequestDto)
    {
        Transfer transfer = transferConverter.toModel(transferRequestDto);
        Transfer performedTransfer = transferService.performTransfer(transfer);
        return transferConverter.toDto(performedTransfer);
    }

    @GetMapping("/statement/{iban}")
    public List<TransferResponseDto> getStatement(
        @PathVariable("iban") String iban,
        @RequestParam("from") LocalDate from,
        @RequestParam("to") LocalDate to) {
        List<Transfer> statement = transferService.getStatement(iban, from, to);
        return transferConverter.toDtos(statement);
    }

    @PostMapping("/favorite")
    @ResponseStatus(HttpStatus.CREATED)
    public FavoriteTransferResponseDto createFavorite(
        @Valid @RequestBody CreateFavoriteTransferRequestDto createFavoriteTransferRequestDto) {
        FavoriteTransfer favoriteTransfer = favoriteTransferConverter.toModel(createFavoriteTransferRequestDto);
        FavoriteTransfer newFavoriteTransfer = transferService.createFavorite(favoriteTransfer);
        return favoriteTransferConverter.toDto(newFavoriteTransfer);
    }

    @DeleteMapping("/favorite/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteFavorite(@PathVariable("id") Long id) {
        transferService.deleteFavoriteById(id);
    }

    @GetMapping("/favorite")

```

```

    public List<FavoriteTransferResponseDto> getAllFavorite() {
        List<FavoriteTransfer> models = transferService.getAllFavorite();
        return favoriteTransferConverter.toDtos(models);
    }
}

TransferRequestDto.java:
package org.example.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
public class TransferRequestDto {

    @Schema(
        description = "IBAN of the account from which money will be sent",
        example = "IBANQEDESSDGPEB3FR9H8QZ2MOT1S58A",
        requiredMode = Schema.RequiredMode.REQUIRED
    )
    @NotBlank(message = "IBAN is required to perform the transfer")
    @Size(min = 32, max = 32)
    private String ibanFrom;

    @Schema(
        description = "IBAN of the account to which money will be sent",
        example = "IBANQEDESSDGPEB3FR9H8QZ2MOT1S58A",
        requiredMode = Schema.RequiredMode.REQUIRED
    )
    @NotBlank(message = "IBAN is required to perform the transfer")
    @Size(min = 32, max = 32)
    private String ibanTo;

    @Schema(
        description = "Amount of money which will be sent",
        example = "1234.00",
        requiredMode = Schema.RequiredMode.REQUIRED
    )
    @NotNull(message = "Sent amount is required to perform the transfer")
    @Positive(message = "Sent amount cannot be less or equals to 0")
    @Digits(integer = 18, fraction = 2, message = "The amount must contain no more than 2 decimal places")
    private BigDecimal sentAmount;
}

TransferResponseDto.java:
package org.example.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Getter;
import lombok.Setter;
import org.example.model.AccountCurrency;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Getter
@Setter
public class TransferResponseDto {

```



```

    @Schema(description = "IBAN of the account from which money was sent", example =
"IBANQEDESSDGPEB3FR9H8QZ2MOT1S58A")
    private String ibanFrom;

    @Schema(description = "Balance of the account from which money was sent", example = "1234.00")
    private BigDecimal senderBalance;

    @Schema(description = "Account currency of the account from which money was sent")
    private AccountCurrency senderCurrency;

    @Schema(description = "Amount of money which was sent", example = "1234.00")
    private BigDecimal amount;

    @Schema(description = "IBAN of the account to which money was sent", example =
"IBANQEDESSDGPEB3FR9H8QZ2MOT1S58A")
    private String ibanTo;

    @Schema(description = "Balance of the account to which money was sent", example = "1234.00")
    private BigDecimal receiverBalance;

    @Schema(description = "Account currency of the account to which money was sent")
    private AccountCurrency receiverCurrency;

    @Schema(description = "Date and time of the transfer")
    private LocalDateTime dateTime;
}

```

```

AccountSettingsDto:
package org.example.dto;

```

```

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.Setter;

```

```

import java.math.BigDecimal;

```

```

@Getter
@Setter
@AllArgsConstructor
public class AccountSettingsDto {

    @Schema(description = "Monthly spending limit for the account",
        example = "5000.00",
        defaultValue = "null"
    )
    private BigDecimal monthlyLimit;

    @Schema(description = "Whether SMS notifications are enabled for this account",
        example = "true",
        defaultValue = "false"
    )
    private Boolean smsNotificationsEnabled;
}

```

```

AccountServiceImpl.java:
package org.example.service;

```

```

import lombok.RequiredArgsConstructor;
import org.example.converter.AccountEntityAndModelConverter;

```

```

import org.example.entity.AccountEntity;
import org.example.model.AccountSettings;
import org.example.model.AccountStatus;
import org.example.exception.EntityNotFoundException;
import org.example.model.Account;
import org.example.repository.AccountRepository;
import org.example.util.IbanGenerator;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@RequiredArgsConstructor
public class AccountServiceImpl implements AccountService {

    private final AccountRepository accountRepository;
    private final AccountEntityAndModelConverter accountConverter;

    @Override
    public Account create(Account account) {
        account.setStatus(AccountStatus.ACTIVE);
        account.setIban(IbanGenerator.generateIban());
        setDefaultSettingsIfNeeded(account);
        AccountEntity entity = accountConverter.toEntity(account);
        entity.getSettings().setAccount(entity);
        AccountEntity newEntity = accountRepository.save(entity);
        return accountConverter.toModel(newEntity);
    }

    @Transactional(readOnly = true)
    @Override
    public List<Account> getAll() {
        List<AccountEntity> accountEntities = accountRepository.findAllWithSettings();
        return accountConverter.toModels(accountEntities);
    }

    @Cacheable(value = "balance")
    @Transactional(readOnly = true)
    @Override
    public Account getByIban(String iban) {
        AccountEntity accountEntity = accountRepository.findByIban(iban)
            .orElseThrow(() -> new EntityNotFoundException(
                "Account not found with IBAN: %s".formatted(iban)));
        return accountConverter.toModel(accountEntity);
    }

    private void setDefaultSettingsIfNeeded(Account account) {
        if (account.getSettings() == null) {
            AccountSettings defaultSettings = new AccountSettings();
            defaultSettings.setSmsNotificationsEnabled(false);
            account.setSettings(defaultSettings);
        } else if (account.getSettings().getSmsNotificationsEnabled() == null) {
            account.getSettings().setSmsNotificationsEnabled(false);
        }
    }
}

```

CurrencyRateServiceImpl.java:
package org.example.service;

```

import lombok.RequiredArgsConstructor;
import org.example.converter.CurrencyEntityAndModelConverter;
import org.example.entity.CurrencyRateEntity;
import org.example.exception.EntityNotFoundException;
import org.example.model.AccountCurrency;
import org.example.model.CurrencyRate;
import org.example.repository.CurrencyRateRepository;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.util.List;

@Service
@RequiredArgsConstructor
public class CurrencyRateServiceImpl implements CurrencyRateService {

    private final CurrencyRateRepository currencyRateRepository;
    private final CurrencyEntityAndModelConverter currencyRateConverter;

    @Override
    public CurrencyRate getTodayRate(AccountCurrency currency) {
        CurrencyRateEntity entity = currencyRateRepository.getByCurrencyAndDate(currency, LocalDate.now())
            .orElseThrow(() -> new EntityNotFoundException("Currency rate not found:
%s".formatted(currency.toString())));
        return currencyRateConverter.toModel(entity);
    }

    @Override
    public void saveAll(List<CurrencyRate> currencyRates) {
        List<CurrencyRateEntity> entities = currencyRateConverter.toEntities(currencyRates);
        currencyRateRepository.saveAll(entities);
    }
}

```

FetchCurrencyRatesService.java:

```
package org.example.service;
```

```

import lombok.RequiredArgsConstructor;
import org.example.model.CurrencyRate;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class FetchCurrencyRatesService {

    private final NationalBankService nationalBankService;
    private final CurrencyRateService currencyRateService;

    public void fetchAndSaveDailyRates() {
        List<CurrencyRate> currencyRates = nationalBankService.fetchDailyRates();
        currencyRateService.saveAll(currencyRates);
    }
}

```

NationalBankService.java:

```
package org.example.service;
```

```

import lombok.RequiredArgsConstructor;
import org.example.client.NationalBankClient;

```

```

import org.example.converter.CurrencyModelAndDtoConverter;
import org.example.dto.CurrencyRateResponseDto;
import org.example.model.AccountCurrency;
import org.example.model.CurrencyRate;
import org.springframework.stereotype.Service;

import java.util.Arrays;
import java.util.List;

@Service
@RequiredArgsConstructor
public class NationalBankService {

    private static final int DAILY_CURRENCY_RATE_PERIODICITY = 0;

    private final NationalBankClient nationalBankClient;
    private final CurrencyModelAndDtoConverter currencyRateConverter;

    public List<CurrencyRate> fetchDailyRates() {
        List<CurrencyRateResponseDto> currencyRateResponseDtos =
            nationalBankClient.getRates(DAILY_CURRENCY_RATE_PERIODICITY)
                .stream()
                .filter(dto -> Arrays.stream(AccountCurrency.values())
                    .anyMatch(a -> a.toString().equals(dto.getCurrency()))))
                .toList();
        return currencyRateConverter.toModels(currencyRateResponseDtos);
    }
}

```

TransferServiceImpl.java:

```

package org.example.service;

import lombok.RequiredArgsConstructor;
import org.example.annotation.LogExecutionTime;
import org.example.converter.FavoriteTransferEntityAndModelConverter;
import org.example.converter.TransferEntityAndModelConverter;
import org.example.entity.AccountEntity;
import org.example.entity.FavoriteTransferEntity;
import org.example.entity.TransferEntity;
import org.example.exception.EntityNotFoundException;
import org.example.model.*;
import org.example.repository.AccountRepository;
import org.example.repository.FavoriteTransferRepository;
import org.example.repository.TransferRepository;
import org.jspecify.annotations.NonNull;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Objects;

```

```

@Service
@RequiredArgsConstructor
public class TransferServiceImpl implements TransferService {

    private static final int AMOUNT_SCALE = 2;

    private final AccountRepository accountRepository;

```

```

private final TransferRepository transferRepository;
private final FavoriteTransferRepository favoriteTransferRepository;
private final TransferEntityAndModelConverter transferConverter;
private final FavoriteTransferEntityAndModelConverter favoriteTransferConverter;
private final CurrencyRateService currencyRateService;

@Override
@Transactional
@LogExecutionTime
public Transfer performTransfer(Transfer transfer) {
    AccountEntity sender = accountRepository.findByIbanWithLock(transfer.getIbanFrom())
        .orElseThrow(() -> new EntityNotFoundException("Account not found with IBAN:
%s".formatted(transfer.getIbanFrom())));
    AccountEntity receiver = accountRepository.findByIbanWithLock(transfer.getIbanTo())
        .orElseThrow(() -> new EntityNotFoundException("Account not found with IBAN:
%s".formatted(transfer.getIbanTo())));

    validateBeforeTransfer(sender, receiver, transfer);

    BigDecimal receivedAmount = convertAmountToReceiverCurrency(
        transfer.getSentAmount(), sender.getCurrency(), receiver.getCurrency());

    BigDecimal senderFinalBalance = sender.getBalance().subtract(transfer.getSentAmount());
    BigDecimal receiverFinalBalance = receiver.getBalance().add(receivedAmount);

    sender.setBalance(senderFinalBalance);
    receiver.setBalance(receiverFinalBalance);

    transfer.setSenderBalance(senderFinalBalance);
    transfer.setReceiverBalance(receiverFinalBalance);
    transfer.setSenderCurrency(sender.getCurrency());
    transfer.setReceiverCurrency(receiver.getCurrency());

    TransferEntity entity = transferConverter.toEntity(transfer);
    TransferEntity performedTransferEntity = transferRepository.save(entity);
    return transferConverter.toModel(performedTransferEntity);
}

@Override
public List<Transfer> getStatement(String iban, LocalDate from, LocalDate to) {
    if (accountRepository.findByIban(iban).isEmpty()) {
        throw new EntityNotFoundException("Account not found with IBAN: %s".formatted(iban));
    }

    LocalDateTime fromDateTime = from.atStartOfDay();
    LocalDateTime toDateTime = to.atTime(23, 59, 59);

    List<TransferEntity> transferEntities = transferRepository.findByIbanAndDateRange(iban, fromDateTime,
toDateTime);
    return transferConverter.toModels(transferEntities);
}

@Override
public FavoriteTransfer createFavorite(FavoriteTransfer favoriteTransfer) {
    FavoriteTransferEntity entity = favoriteTransferConverter.toEntity(favoriteTransfer);
    FavoriteTransferEntity newEntity = favoriteTransferRepository.save(entity);
    return favoriteTransferConverter.toModel(newEntity);
}

@Override
public void deleteFavoriteById(Long id) {
    favoriteTransferRepository.deleteById(id);
}

```

```

@Override
public List<FavoriteTransfer> getAllFavorite() {
    List<FavoriteTransferEntity> entities = favoriteTransferRepository.findAll();
    return favoriteTransferConverter.toModels(entities);
}

private void validateBeforeTransfer(AccountEntity sender, AccountEntity receiver, Transfer transfer) {
    if (sender.getStatus() != AccountStatus.ACTIVE) {
        throw new IllegalArgumentException("Impossible to perform transfer: sender account isn't active");
    } else if (receiver.getStatus() != AccountStatus.ACTIVE) {
        throw new IllegalArgumentException("Impossible to perform transfer: receiver account isn't active");
    }
    if (Objects.equals(sender.getIban(), receiver.getIban())) {
        throw new IllegalArgumentException(
            "Impossible to perform transfer: receiver IBAN(%s) is equals to sender IBAN".formatted(transfer.getIbanFrom()));
    } else if (transfer.getSentAmount().compareTo(sender.getBalance()) > 0) {
        throw new IllegalArgumentException(
            "Impossible to perform transfer: current balance(%s) is less than the transfer amount(%s)".formatted(sender.getBalance(), transfer.getSentAmount()));
    }
}

private @NonNull BigDecimal convertAmountToReceiverCurrency(
    BigDecimal sentAmount,
    AccountCurrency senderAccountCurrency,
    AccountCurrency receiverAccountCurrency) {
    if (senderAccountCurrency == receiverAccountCurrency) {
        return sentAmount;
    }

    BigDecimal bynSentAmount;
    if (senderAccountCurrency != AccountCurrency.BYN) {
        bynSentAmount = convertSendAmountToByn(sentAmount, senderAccountCurrency);
    } else {
        bynSentAmount = sentAmount;
    }

    if (receiverAccountCurrency != AccountCurrency.BYN) {
        return convertSendAmountToReceiverCurrency(receiverAccountCurrency, bynSentAmount);
    } else {
        return bynSentAmount;
    }
}

private @NonNull BigDecimal convertSendAmountToByn(BigDecimal sentAmount, AccountCurrency senderAccountCurrency) {
    CurrencyRate senderCurrencyRate = currencyRateService.getTodayRate(senderAccountCurrency);
    return sentAmount
        .multiply(senderCurrencyRate.getRate())
        .divide(new BigDecimal(senderCurrencyRate.getScale()), AMOUNT_SCALE, RoundingMode.HALF_UP);
}

private @NonNull BigDecimal convertSendAmountToReceiverCurrency(AccountCurrency receiverAccountCurrency, BigDecimal bynSentAmount) {
    CurrencyRate receiverCurrencyRate = currencyRateService.getTodayRate(receiverAccountCurrency);
    return bynSentAmount
        .multiply(new BigDecimal(receiverCurrencyRate.getScale()))
        .divide(receiverCurrencyRate.getRate(), AMOUNT_SCALE, RoundingMode.HALF_UP);
}
}

```

```

Account.java:
package org.example.model;

import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
public class Account {

    private Long id;
    private BigDecimal balance;
    private AccountCurrency currency;
    private AccountStatus status;
    private String iban;
    private AccountSettings settings;
}

AccountCurrency.java:
package org.example.model;

public enum AccountCurrency {
    EUR, USD, BYN, RUB
}

AccountSettings.java:
package org.example.model;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class AccountSettings {

    private BigDecimal monthlyLimit;

    private Boolean smsNotificationsEnabled;
}

AccountStatus.java:
package org.example.model;

public enum AccountStatus {
    ACTIVE, BLOCKED
}

CurrencyRate.java:
package org.example.model;

```

```

import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

@Getter
@Setter
public class CurrencyRate {

    private LocalDate date;
    private AccountCurrency currency;
    private Integer scale;
    private BigDecimal rate;
}

```

FavoriteTransfer.java:

```

package org.example.model;

```

```

import lombok.Builder;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
@Builder
public class FavoriteTransfer {

    private Long id;
    String ibanFrom;
    String ibanTo;
    BigDecimal amount;
}

```

Transfer.java:

```

package org.example.model;

```

```

import lombok.Builder;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Getter
@Setter
@Builder
public class Transfer {

    private String ibanFrom;
    private BigDecimal senderBalance;
    private AccountCurrency senderCurrency;
    private BigDecimal sentAmount;
    private String ibanTo;
    private BigDecimal receiverBalance;
    private AccountCurrency receiverCurrency;
    private LocalDateTime dateTime;
}

```



```

EntityToModelConverter.java:
package org.example.converter;

import java.util.List;

public interface EntityToModelConverter<E, M> {

    M toModel(E entity);

    default List<M> toModels(List<E> entities) {
        return entities.stream()
            .map(this::toModel)
            .toList();
    }
}

```

```

ModelToEntityConverter.java:
package org.example.converter;

import java.util.List;

public interface ModelToEntityConverter <M, E> {

    E toEntity(M model);

    default List<E> toEntities(List<M> models) {
        return models.stream()
            .map(this::toEntity)
            .toList();
    }
}

```

```

ModelToDtoConverter.java:
package org.example.converter;

import java.util.List;

public interface ModelToDtoConverter <M, D> {

    D toDto(M model);

    default List<D> toDtos(List<M> models) {
        return models.stream()
            .map(this::toDto)
            .toList();
    }
}

```

```

DtoToModelConverter.java:
package org.example.converter;

import java.util.List;

public interface DtoToModelConverter <D, M> {

    M toModel(D dto);

    default List<M> toModels(List<D> dtos) {
        return dtos.stream()

```

```

        .map(this::toModel)
        .toList();
    }
}

```

AccountEntity.java:
package org.example.entity;

```

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import org.example.model.AccountCurrency;
import org.example.model.AccountStatus;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

```

```

import java.math.BigDecimal;
import java.time.LocalDateTime;

```

```

@Entity
@Table(name = "accounts")
@Getter
@Setter
public class AccountEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private BigDecimal balance;

    @Enumerated(EnumType.STRING)
    private AccountCurrency currency;

    @Enumerated(EnumType.STRING)
    private AccountStatus status;

    private String iban;

    @OneToOne(mappedBy = "account", cascade = CascadeType.ALL, optional = false)
    private AccountSettingsEntity settings;

    @CreationTimestamp
    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;
}

```

AccountSettingsEntity.java:
package org.example.entity;

```

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.MapId;
import jakarta.persistence.OneToOne;

```

```

import jakarta.persistence.Table;
import lombok.*;

import java.math.BigDecimal;

@Entity
@Table(name = "account_settings")
@Getter
@Setter
@NoArgsConstructor
public class AccountSettingsEntity {

    public AccountSettingsEntity(BigDecimal monthlyLimit, Boolean smsNotificationsEnabled) {
        this.monthlyLimit = monthlyLimit;
        this.smsNotificationsEnabled = smsNotificationsEnabled;
    }

    @Id
    @Column(name = "account_id")
    private Long accountId;

    @OneToOne(fetch = FetchType.LAZY)
    @MapsId
    @JoinColumn(name = "account_id")
    private AccountEntity account;

    @Column(name = "monthly_limit")
    private BigDecimal monthlyLimit;

    @Column(name = "sms_notifications_enabled")
    private Boolean smsNotificationsEnabled;
}

```

CurrencyRateEntity.java:
 package org.example.entity;

```

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Getter;
import lombok.Setter;
import org.example.model.AccountCurrency;
import org.hibernate.annotations.CreationTimestamp;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "currency_rates")
@Getter
@Setter
public class CurrencyRateEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

```

private LocalDate date;

@Enumerated(EnumType.STRING)
private AccountCurrency currency;

private Integer scale;

private BigDecimal rate;

@CreationTimestamp
@Column(name = "created_at")
private LocalDateTime createdAt;
}

```

FavoriteTransferEntity.java:

```

package org.example.entity;

import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "favorite_transfers")
@Getter
@Setter
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class FavoriteTransferEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "iban_from")
    private String ibanFrom;

    @Column(name = "iban_to")
    private String ibanTo;

    private BigDecimal amount;

    @CreationTimestamp
    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;
}

```

TransferEntity.java:

```

package org.example.entity;

import jakarta.persistence.*;
import lombok.Getter;

```

```

import lombok.Setter;
import org.example.model.AccountCurrency;
import org.hibernate.annotations.CreationTimestamp;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "transfers")
@Getter
@Setter
public class TransferEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "iban_from")
    private String ibanFrom;

    @Column(name = "sender_balance")
    private BigDecimal senderBalance;

    @Enumerated(EnumType.STRING)
    @Column(name = "sender_currency")
    private AccountCurrency senderCurrency;

    private BigDecimal amount;

    @Column(name = "iban_to")
    private String ibanTo;

    @Enumerated(EnumType.STRING)
    @Column(name = "receiver_currency")
    private AccountCurrency receiverCurrency;

    @Column(name = "receiver_balance")
    private BigDecimal receiverBalance;

    @CreationTimestamp
    @Column(name = "created_at")
    private LocalDateTime createdAt;
}

```

FetchAndSaveDailyCurrencyRatesIT.java:

```
package org.example.integration;
```

```

import com.github.tomakehurst.wiremock.core.WireMockConfiguration;
import com.github.tomakehurst.wiremock.junit5.WireMockExtension;
import org.example.entity.CurrencyRateEntity;
import org.example.model.AccountCurrency;
import org.example.repository.CurrencyRateRepository;
import org.example.scheduler.CurrencyRateScheduler;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.RegisterExtension;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;

```

```

import org.testcontainers.junit.jupiter.Testcontainers;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

import static com.github.tomakehurst.wiremock.client.WireMock.*;
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

@Testcontainers
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class FetchAndSaveDailyCurrencyRatesIT {

    @SuppressWarnings("resource")
    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test");

    @RegisterExtension
    static WireMockExtension nationalBankMock = WireMockExtension.newInstance()
        .options(WireMockConfiguration.wireMockConfig().dynamicPort())
        .build();

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("national-bank-client.url", nationalBankMock::baseUrl);
        registry.add("national-bank-client.currency-rates-endpoint", () -> "/ExRates");

        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);

        registry.add("spring.task.scheduling.enabled", () -> "false");
    }

    private final CurrencyRateRepository currencyRateRepository;
    private final CurrencyRateScheduler currencyRateScheduler;

    @Autowired
    public FetchAndSaveDailyCurrencyRatesIT(
        CurrencyRateRepository currencyRateRepository,
        CurrencyRateScheduler currencyRateScheduler) {
        this.currencyRateRepository = currencyRateRepository;
        this.currencyRateScheduler = currencyRateScheduler;
    }

    @BeforeEach
    void cleanDatabase() {
        currencyRateRepository.deleteAll();
    }

    @Test
    void fetchAndSaveRates_Success() {
        nationalBankMock.stubFor(get(urlPathEqualTo("/ExRates"))
            .withQueryParam("periodicity", equalTo("0"))
            .willReturn(aResponse()
                .withStatus(200)
                .withHeader("Content-Type", "application/json")
                .withBody(MOCK_RESPONSE1)));
    }

```

```

currencyRateScheduler.fetchAndSaveDailyRates();

nationalBankMock.verify(1, getRequestedFor(urlPathEqualTo("/ExRates"))
    .withQueryParam("periodicity", equalTo("0")));

List<CurrencyRateEntity> savedRates = currencyRateRepository.findAll();

assertThat(savedRates).hasSize(3);

assertThat(savedRates)
    .extracting("currency")
    .containsExactlyInAnyOrder(
        AccountCurrency.EUR,
        AccountCurrency.USD,
        AccountCurrency.RUB
    );

CurrencyRateEntity usdRate = savedRates.stream()
    .filter(r -> r.getCurrency() == AccountCurrency.USD)
    .findFirst()
    .orElseThrow();
assertThat(usdRate.getRate()).isEqualByComparingTo(new BigDecimal("2.8230"));
assertThat(usdRate.getScale()).isEqualTo(1);
assertThat(usdRate.getDate()).isEqualTo(LocalDate.of(2026, 5, 2));

CurrencyRateEntity rubRate = savedRates.stream()
    .filter(r -> r.getCurrency() == AccountCurrency.RUB)
    .findFirst()
    .orElseThrow();
assertThat(rubRate.getRate()).isEqualByComparingTo(new BigDecimal("3.7683"));
assertThat(rubRate.getScale()).isEqualTo(100);
assertThat(rubRate.getDate()).isEqualTo(LocalDate.of(2026, 5, 2));

CurrencyRateEntity eurRate = savedRates.stream()
    .filter(r -> r.getCurrency() == AccountCurrency.EUR)
    .findFirst()
    .orElseThrow();
assertThat(eurRate.getRate()).isEqualByComparingTo(new BigDecimal("3.3018"));
assertThat(eurRate.getScale()).isEqualTo(1);
assertThat(eurRate.getDate()).isEqualTo(LocalDate.of(2026, 5, 2));
}

@Test
void whenApiReturnsError_ShouldHandleGracefully() {
    nationalBankMock.stubFor(get(urlPathEqualTo("/ExRates"))
        .withQueryParam("periodicity", equalTo("0"))
        .willReturn(aResponse()
            .withStatus(500)
            .withBody("{\"error\": \"Internal Server Error\"}")));

    assertThatThrownBy(currencyRateScheduler::fetchAndSaveDailyRates);

    List<CurrencyRateEntity> savedRates = currencyRateRepository.findAll();
    assertThat(savedRates).isEmpty();

    nationalBankMock.verify(1, getRequestedFor(urlPathEqualTo("/ExRates"))
        .withQueryParam("periodicity", equalTo("0")));
}

@Test
void whenApiReturnsEmptyList_ShouldSaveNothing() {
    nationalBankMock.stubFor(get(urlPathEqualTo("/ExRates"))
        .withQueryParam("periodicity", equalTo("0"))

```

```

        .willReturn(aResponse()
            .withStatus(200)
            .withHeader("Content-Type", "application/json")
            .withBody("[]"));

currencyRateScheduler.fetchAndSaveDailyRates();

List<CurrencyRateEntity> savedRates = currencyRateRepository.findAll();
assertThat(savedRates).isEmpty();

nationalBankMock.verify(1, getRequestedFor(urlPathEqualTo("/ExRates"))
    .withQueryParam("periodicity", equalTo("0")));
}

@Test
void whenApiReturnsOnlyUnsupportedCurrencies_ShouldSaveNothing() {
    nationalBankMock.stubFor(get(urlPathEqualTo("/ExRates"))
        .withQueryParam("periodicity", equalTo("0"))
        .willReturn(aResponse()
            .withStatus(200)
            .withHeader("Content-Type", "application/json")
            .withBody(MOCK_RESPONSE2)));

    currencyRateScheduler.fetchAndSaveDailyRates();

    List<CurrencyRateEntity> savedRates = currencyRateRepository.findAll();
    assertThat(savedRates).isEmpty();
}

private final String MOCK_RESPONSE1 = ""
[
    {
        "Cur_ID": 440,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "AUD",
        "Cur_Scale": 1,
        "Cur_Name": "Австралийский доллар",
        "Cur_OfficialRate": 2.0187
    },
    {
        "Cur_ID": 510,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "AMD",
        "Cur_Scale": 1000,
        "Cur_Name": "Армянских драмов",
        "Cur_OfficialRate": 7.6010
    },
    {
        "Cur_ID": 514,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "BRL",
        "Cur_Scale": 10,
        "Cur_Name": "Бразильских реалов",
        "Cur_OfficialRate": 5.6756
    },
    {
        "Cur_ID": 449,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "UAH",
        "Cur_Scale": 100,
        "Cur_Name": "Гривен",
        "Cur_OfficialRate": 6.4071
    },
]

```



```

{
  "Cur_ID": 450,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "DKK",
  "Cur_Scale": 10,
  "Cur_Name": "Датских крон",
  "Cur_OfficialRate": 4.4185
},
{
  "Cur_ID": 513,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "AED",
  "Cur_Scale": 10,
  "Cur_Name": "Дирхамов ОАЭ",
  "Cur_OfficialRate": 7.6869
},
{
  "Cur_ID": 431,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "USD",
  "Cur_Scale": 1,
  "Cur_Name": "Доллар США",
  "Cur_OfficialRate": 2.8230
},
{
  "Cur_ID": 512,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "VND",
  "Cur_Scale": 100000,
  "Cur_Name": "Донгов",
  "Cur_OfficialRate": 10.7110
},
{
  "Cur_ID": 451,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "EUR",
  "Cur_Scale": 1,
  "Cur_Name": "Евро",
  "Cur_OfficialRate": 3.3018
},
{
  "Cur_ID": 452,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "PLN",
  "Cur_Scale": 10,
  "Cur_Name": "Злотых",
  "Cur_OfficialRate": 7.7632
},
{
  "Cur_ID": 508,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "JPY",
  "Cur_Scale": 100,
  "Cur_Name": "Иен",
  "Cur_OfficialRate": 1.7661
},
{
  "Cur_ID": 511,
  "Date": "2026-05-02T00:00:00",
  "Cur_Abbreviation": "INR",
  "Cur_Scale": 100,
  "Cur_Name": "Индийских рупий",
  "Cur_OfficialRate": 2.9764
}

```

```

    },
    {
        "Cur_ID": 461,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "IRR",
        "Cur_Scale": 100000,
        "Cur_Name": "Иранских риалов",
        "Cur_OfficialRate": 0.2145
    },
    {
        "Cur_ID": 453,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "ISK",
        "Cur_Scale": 100,
        "Cur_Name": "Исландских крон",
        "Cur_OfficialRate": 2.3068
    },
    {
        "Cur_ID": 371,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "CAD",
        "Cur_Scale": 1,
        "Cur_Name": "Канадский доллар",
        "Cur_OfficialRate": 2.0619
    },
    {
        "Cur_ID": 462,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "CNY",
        "Cur_Scale": 10,
        "Cur_Name": "Китайских юаней",
        "Cur_OfficialRate": 4.1320
    },
    {
        "Cur_ID": 394,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "KWD",
        "Cur_Scale": 1,
        "Cur_Name": "Кувейтский динар",
        "Cur_OfficialRate": 9.1715
    },
    {
        "Cur_ID": 454,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "MDL",
        "Cur_Scale": 10,
        "Cur_Name": "Молдавских леев",
        "Cur_OfficialRate": 1.6398
    },
    {
        "Cur_ID": 448,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "NZD",
        "Cur_Scale": 1,
        "Cur_Name": "Новозеландский доллар",
        "Cur_OfficialRate": 1.6523
    },
    {
        "Cur_ID": 455,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "NOK",
        "Cur_Scale": 10,
        "Cur_Name": "Норвежских крон",
    }

```

```

    "Cur_OfficialRate": 3.0306
  },
  {
    "Cur_ID": 456,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "RUB",
    "Cur_Scale": 100,
    "Cur_Name": "Российских рублей",
    "Cur_OfficialRate": 3.7683
  },
  {
    "Cur_ID": 457,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "XDR",
    "Cur_Scale": 1,
    "Cur_Name": "СДР (Специальные права заимствования)",
    "Cur_OfficialRate": 3.8656
  },
  {
    "Cur_ID": 421,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "SGD",
    "Cur_Scale": 1,
    "Cur_Name": "Сингапурский доллар",
    "Cur_OfficialRate": 2.2094
  },
  {
    "Cur_ID": 458,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "KGS",
    "Cur_Scale": 100,
    "Cur_Name": "Сомов",
    "Cur_OfficialRate": 3.2289
  },
  {
    "Cur_ID": 459,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "KZT",
    "Cur_Scale": 1000,
    "Cur_Name": "Тенге",
    "Cur_OfficialRate": 6.1110
  },
  {
    "Cur_ID": 460,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "TRY",
    "Cur_Scale": 10,
    "Cur_Name": "Турецких лир",
    "Cur_OfficialRate": 0.6264
  },
  {
    "Cur_ID": 429,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "GBP",
    "Cur_Scale": 1,
    "Cur_Name": "Фунт стерлингов",
    "Cur_OfficialRate": 3.8128
  },
  {
    "Cur_ID": 463,
    "Date": "2026-05-02T00:00:00",
    "Cur_Abbreviation": "CZK",
    "Cur_Scale": 100,

```

```

        "Cur_Name": "Чешских крон",
        "Cur_OfficialRate": 13.5462
    },
    {
        "Cur_ID": 464,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "SEK",
        "Cur_Scale": 10,
        "Cur_Name": "Шведских крон",
        "Cur_OfficialRate": 3.0414
    },
    {
        "Cur_ID": 426,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "CHF",
        "Cur_Scale": 1,
        "Cur_Name": "Швейцарский франк",
        "Cur_OfficialRate": 3.5730
    }
]
"".replaceAll("\\s", "");

String MOCK_RESPONSE2 = ""
[
    {
        "Cur_ID": 510,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "AMD",
        "Cur_Scale": 1000,
        "Cur_Name": "Армянских драмов",
        "Cur_OfficialRate": 7.6010
    },
    {
        "Cur_ID": 514,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "BRL",
        "Cur_Scale": 10,
        "Cur_Name": "Бразильских реалов",
        "Cur_OfficialRate": 5.6756
    },
    {
        "Cur_ID": 449,
        "Date": "2026-05-02T00:00:00",
        "Cur_Abbreviation": "UAH",
        "Cur_Scale": 100,
        "Cur_Name": "Гривен",
        "Cur_OfficialRate": 6.4071
    }
]
"".replaceAll("\\s", "");
}

```

CardRepositoryImpl.java:

```
package com.example.repository;
```

```

import com.example.entity.CardEntity;
import com.example.model.CardStatus;
import lombok.RequiredArgsConstructor;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;

```

```

import java.sql.Timestamp;
import java.time.YearMonth;
import java.util.List;
import java.util.Optional;

@Repository
@RequiredArgsConstructor
public class CardRepositoryImpl implements CardRepository {

    private final JdbcTemplate jdbcTemplate;

    private final RowMapper<CardEntity> cardRowMapper = (rs, _) -> new CardEntity(
        rs.getLong("id"),
        rs.getString("number"),
        rs.getString("holder_name"),
        YearMonth.parse(rs.getString("expiration_date")),
        CardStatus.valueOf(rs.getString("status")),
        rs.getString("iban"),
        rs.getTimestamp("created_at").toLocalDateTime(),
        Optional.ofNullable(rs.getTimestamp("updated_at"))
            .map(Timestamp::toLocalDateTime)
            .orElse(null)
    );

    @Override
    public Optional<CardEntity> getById(Long id) {
        String sql = "SELECT * FROM cards WHERE id = ?";
        return jdbcTemplate.query(sql, cardRowMapper, id)
            .stream()
            .findFirst();
    }

    @Override
    public List<CardEntity> getAll() {
        String sql = "SELECT * FROM cards";
        return jdbcTemplate.query(sql, cardRowMapper);
    }

    @Override
    public CardEntity insert(CardEntity cardEntity) {
        String sql = ""
            + "INSERT INTO cards (number, holder_name, expiration_date, status, iban)"
            + "VALUES (?, ?, ?, ?, ?)"
            + "RETURNING *"
            + ""
            + ";

        return jdbcTemplate.queryForObject(sql, cardRowMapper,
            cardEntity.getNumber(),
            cardEntity.getHolderName(),
            cardEntity.getExpirationDate() != null ? cardEntity.getExpirationDate().toString() : null,
            cardEntity.getStatus() != null ? cardEntity.getStatus().toString() : null,
            cardEntity.getIban()
        );
    }

    @Override
    public Optional<CardEntity> update(Long id, CardEntity cardEntity) {
        String sql = ""
            + "UPDATE cards SET"
            + "    number = COALESCE(?, number),"
            + "    holder_name = COALESCE(?, holder_name),"
            + "    expiration_date = COALESCE(?, expiration_date),"
            + "    status = COALESCE(?, status),

```

```

        iban = COALESCE(?, iban),
        updated_at = CURRENT_TIMESTAMP
    WHERE id = ?
    RETURNING *
    """,

    return jdbcTemplate.query(sql, cardRowMapper,
        cardEntity.getNumber(),
        cardEntity.getHolderName(),
        cardEntity.getExpirationDate() != null ? cardEntity.getExpirationDate().toString() : null,
        cardEntity.getStatus() != null ? cardEntity.getStatus().toString() : null,
        cardEntity.getIban(),
        id
    ).stream()
    .findFirst();
}

@Override
public Optional<CardEntity> deleteById(Long id) {
    String sql = "DELETE FROM cards WHERE id = ? RETURNING *";
    return jdbcTemplate.query(sql, cardRowMapper, id)
        .stream()
        .findFirst();
}
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема алгоритма работы системы